

CS336 作业 2 (系统): 系统与并行

版本 26.1.3

CS336 课程团队

2026 年春季

1 作业概览

在本次作业中, 你将通过动手实践, 学习如何提高单 GPU 训练速度, 并将训练扩展到多个 GPU。

你将实现的内容

1. 基准测试与性能分析框架
2. 激活检查点
3. Flash Attention 2 的 Triton 内核
4. 分布式数据并行训练
5. 优化器状态分片
6. 全分片数据并行训练

代码组织方式

本作业代码及本说明文档发布在 GitHub:

<https://github.com/stanford-cs336/assignment2-systems>

请使用 Git 克隆仓库。如有更新, 我们会通知你, 你可以通过 `git pull` 获取最新版本。

1. `cs336-basics/`: 在本次作业中, 你将分析作业 1 中构建的一些组件的性能。本文件夹包含课程团队的作业 1 参考解答代码, 因此你可以在这里找到 `cs336-basics/pyproject.toml` 和 `cs336-basics/cs336_basics` 包。如果希望使用自己实现的模型, 可以修改根目录中的 `pyproject.toml` 文件, 使其指向你自己的包。
2. `/`: `cs336-systems` 根目录。我们创建了一个名为 `cs336_systems` 的空模块。注意, 其中没有任何代码, 因此你可以从零开始自由实现。
3. `tests/*.py`: 此目录包含你必须通过的所有测试。这些测试会调用 `tests/adapters.py` 中定义的钩子。你需要实现适配器, 将自己的代码连接到测试。编写更多测试和/或修改测试代码有助于调试, 但你的实现应当通过最初提供的测试套件。
4. `README.md`: 此文件进一步说明了预期的目录结构, 并提供了环境配置的基本操作指南。

如何提交

你需要向 Gradescope 提交以下文件:

- `writeup.pdf`: 回答所有书面问题。请对回答进行排版。
- `code.zip`: 包含你编写的全部代码。

运行 `test_and_make_submission.sh` 中的脚本, 生成 `code.zip` 文件。

2 性能分析与基准测试

在作业的第一部分，我们将探索如何优化 Transformer 模型的性能，以最高效地利用 GPU。我们会分析模型，了解前向传播和反向传播中时间与显存的消耗位置；随后使用自定义 GPU 内核优化自注意力运算，使其速度超过常规 PyTorch 实现所能达到的水平。在后续部分，我们将利用多个 GPU，了解如何在一个集群上训练模型。

2.1 性能分析

在实现任何优化之前，先对程序进行性能分析，了解它在哪些地方消耗资源（例如时间和显存），会很有帮助。否则，我们可能优化了并不占用大量时间或显存的模型部分，最终看不到可测量的端到端改进。

我们将实现三种性能评估方式：

1. 使用 Python 标准库对前向传播和反向传播计时，进行简单的端到端基准测试。
2. 使用 NVIDIA Nsight Systems 工具进行计算性能分析，了解这些时间如何分布在 CPU 和 GPU 的不同操作上。
3. 显存分析。

2.1.1 准备工作——导入 basics Transformer 模型

首先确认你能够加载上一次作业中的模型。上一份作业把模型组织成了一个 Python 包，以便之后方便地导入。我们在 `./cs336-basics` 文件夹中加入了模型的参考实现，并在 `pyproject.toml` 文件中指向它。和往常一样调用 `uv run [command]` 时，`uv` 会自动定位这个本地 `cs336-basics` 包。如果希望使用自己的模型实现，可以修改 `pyproject.toml` 文件，使其指向你自己的包。

可以通过以下方式测试能否导入模型：

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
  Built cs336-systems @ file:///path/to/systems/dir
  Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr 7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

现在应该可以使用作业 1 的相关模块了。例如，对于 `model.py`，可以用 `import cs336_basics.model` 导入。

2.1.2 模型规模

在整个作业中，我们将对模型进行基准测试和性能分析，以更好地了解其性能。为了观察性能如何随规模变化，我们将采用并引用下列模型配置。除排行榜任务外，所有模型均使用 10,000 的词表大小、4 的批大小，并改变上下文长度。本次作业（以及后续作业）需要以表格和图形展示大量结果。我们强烈建议你通过代码自动生成报告中的表格，因为在 LaTeX 或 Typst 中手动排版表格可能非常繁琐。

可参考 `pandas.DataFrame.to_latex()` 和 `pandas.DataFrame.to_typst()`，或者自行编写函数，从你偏好的表格表示中生成这些表格。

规模	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

表 1：不同模型规模的配置。这些配置主要基于 GPT-2。

除非另有说明，上下文长度均使用 512。

2.1.3 端到端基准测试

现在我们将实现一个简单的性能评估脚本。后面要测试模型的许多变体，例如改变精度、替换层等。因此，通过命令行参数启用这些变体会很有价值，便于之后运行。

首先采用最简单的方法分析模型性能：对前向传播、反向传播和优化器更新步骤分别计时。由于只测量速度和显存，可以使用随机权重和随机数据。

性能测量有许多微妙之处：一些常见陷阱会导致测到的并不是我们想要的内容。对 GPU 代码进行基准测试时，一个需要注意的问题是 CUDA 调用是异步的。当你调用 CUDA 内核，例如调用 `torch.matmul` 时，PyTorch 函数不会等待矩阵乘法结束，就会将控制权交还给你的代码。这样，GPU 完成矩阵乘法的同时，CPU 可以继续执行并调度新的操作，从而显著提升性能。另一方面，这意味着直接测量 `torch.matmul` 调用返回所需的时间，并不能告诉我们 GPU 实际执行矩阵乘法花了多久。在 PyTorch 中，可以调用 `torch.cuda.synchronize()`，等待所有已调度的 GPU 内核执行完成，从而更准确地测量 CUDA 内核运行时间。这里的“同步”是指 CPU 运行时与 GPU 运行时之间的同步。理解这一点后，让我们编写基本的性能分析基础设施。

题目 (`benchmarking_script`)：基准测试脚本 (4 分)

(a) 编写脚本，对模型的前向传播、反向传播和优化器更新步骤进行基本的端到端基准测试。具体而言，脚本应支持以下功能：

- 根据给定的超参数（例如层数）初始化模型。
- 生成一批随机数据。
- 先执行 w 个预热步骤（在开始计时之前），再测量 n 个步骤的执行时间。根据参数，测量范围可以是仅前向传播、前向加反向传播，或者前向加反向传播再加优化器更新。计时可以使用 Python 的 `timeit` 模块，例如使用 `timeit` 函数，或使用 `timeit.default_timer()`；后者提供系统最高分辨率的时钟，因此作为基准测试的默认选择，比 `time.time()` 更合适。

- 每个步骤结束后调用 `torch.cuda.synchronize()`。

提交内容：一个脚本：根据给定超参数初始化 basics Transformer 模型，创建一批随机数据，并分别测量仅前向传播、前向加反向传播，以及包含优化器更新的完整训练步骤。

(b) 对第 2.1.2 节所述的各模型规模，测量前向传播、反向传播和优化器更新的时间。使用 5 个预热步骤，并对 10 个测量步骤的耗时计算平均值和标准差。一次前向传播需要多久？反向传播呢？各次测量差异很大，还是标准差较小？

提交内容：用 1–2 句话回答，并给出测量时间。

(c) 基准测试的一个常见问题是没有执行预热步骤。不进行预热，重新完成分析。这会如何影响结果？你认为为什么会发生这种情况？再尝试使用 1 个或 2 个预热步骤运行脚本。为什么结果仍可能不同？

提交内容：用 2–3 句话回答。

2.1.4 Nsight Systems 性能分析器

端到端基准测试不能告诉我们，模型在前向和反向传播中究竟把时间和显存消耗在哪里，因此也无法揭示具体的优化机会。为了了解程序在各个组件（例如函数）上耗费的时间，我们可以使用性能分析器。执行性能分析器通过在函数开始和结束时插入监测点来对代码进行插桩，因此能够给出函数级的详细执行统计，例如调用次数、平均耗时、在该函数上累计花费的时间等。

标准 Python 性能分析器（例如 CProfile）不能分析 CUDA 内核，因为这些内核在 GPU 上异步执行。幸运的是，NVIDIA 提供了可通过命令行工具 `nsys` 使用的性能分析器。建议通过包管理器，或其[下载页面](#)的安装程序，获取较新的版本。在本部分中，你将使用 `nsys` 分析 Transformer 模型的运行时间。

`nsys` 的使用很直接：运行上一节的 Python 脚本时，在命令前加上 `nsys profile`。例如，对 `benchmark.py` 进行基本分析：

```
$ uv run nsys profile -- python benchmark.py
```

随后可以在本地机器上使用 [NVIDIA Nsight Systems 桌面应用](#) 查看分析结果。在轨迹的 CUDA API 行中选中某个 CUDA API 调用（位于 CPU 上），会高亮 CUDA HW 行中所有对应的内核执行（位于 GPU 上）。

更全面的性能分析命令可以写成：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx --pytorch=functions-trace,autograd-shapes-nvtx  
--cudabacktrace=all --python-backtrace=cuda --gpu-metrics-devices=0 -- python benchmark.py
```

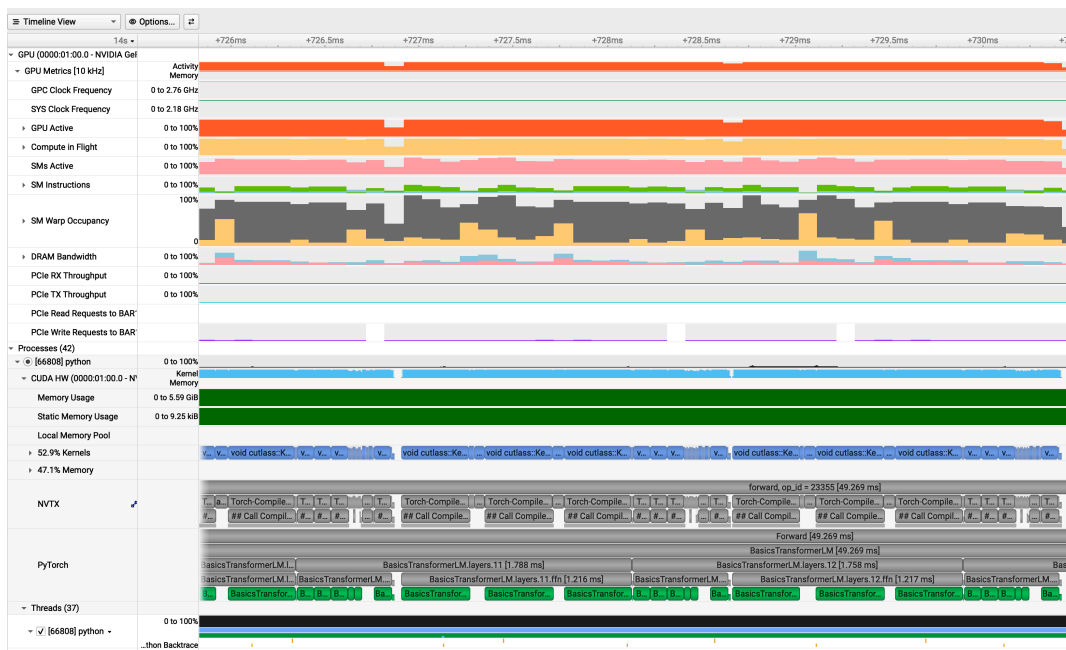


图 1: 详细的 Nsight Systems 执行轨迹。

在这个例子中，`--trace` 指定记录哪些 API；`--pytorch` 在模块调用和 `autograd` 过程中插入 NVTX 标签；`--cudabacktrace` 和 `--python-backtrace` 提供更完善的调用回溯，以了解某个内核由代码中的哪个位置发起；`--gpu-metrics-devices` 指定测量哪个 GPU 的利用率。

为运行过程加入性能分析并非没有代价，它会使运行整体变慢。通常，只启用本次分析所需要的功能更合适。具体而言，如果不需要调用回溯，可以去掉 `--cudabacktrace=all` 和 `--python-backtrace=cuda`，因为它们的开销特别大。

我们鼓励你尝试 `nsys profile` 的各种[命令行选项](#)，了解它的能力。也可以给代码添加 NVTX 范围标记；它们会在轨迹的 NVTX 行中显示为区块，包含相应的全部 CUDA API 调用和关联内核执行。特别是，应当使用 **NVTX 范围**来忽略基准测试脚本中的预热步骤，即在分析时对 NVTX 标签应用 `--nvtx-capture` 过滤器。你还可以分离出负责模型前向、反向传播的内核，甚至通过以下方式标注实现，分离出自注意力层不同部分对应的内核：

```
...
import torch.cuda.nvtx as nvtx
@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, 掩码
):
    ...
    with nvtx.range("computing attention scores"):
        ... # 计算 Q 与 K 之间的注意力分数
    with nvtx.range("computing softmax"):
        ... # 计算注意力分数的 softmax
    with nvtx.range("final matmul"):
        ... # 计算输出投影
```

```
return ...
```

在基准测试脚本中，可以用以下方式将原实现替换为带标注的版本：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

最后需要注意，`torch.compile` 可能使时间和资源难以归因到具体代码部分。为了正确地把时间和资源使用量归因到源代码的各个部分，你可能需要针对不同部分添加或移除 `torch.compile` 包装和 NVTX 标注。

题目 (`nsys_profile`)：Nsight Systems 性能分析 (5 分)

使用 `nsys` 分析前向传播、反向传播和优化器更新。自行从表 1 选择两种模型规模，再选择三个大于 128、且为 2 的幂的上下文长度，其中最大的长度应当是显存能够容纳的最长上下文。请选择你认为最值得观察的组合。对每份分析结果，回答以下问题：

(a) 前向传播的总耗时是多少？它与之前使用 Python 标准库测得的结果一致吗？

提交内容：用 1-2 句话回答。

(b) 前向传播期间，哪个 CUDA 内核累计占用的 GPU 时间最多？在模型的一次前向传播中，这个内核会被调用多少次？同时执行前向和反向传播时，耗时最多的还是这个内核吗？提示：查看“Stats System View”下的“CUDA GPU Kernel Summary”，并使用 NVTX 范围进行过滤，以确定模型的哪些部分对应哪些内核。

提交内容：用 1-2 句话回答。

(c) 尽管绝大多数 FLOP 都发生在矩阵乘法中，你仍会发现一些其他内核占据了不可忽略的总体运行时间。除矩阵乘法外，你观察到前向传播中还有哪些内核占用了相当比例的 CUDA 运行时间？

提交内容：用 1-2 句话回答。

(d) 使用你实现的 AdamW，分析一个完整训练步骤，即像实际训练一样执行前向传播、计算损失、执行反向传播，最后执行优化器更新。与推理（仅前向传播）相比，矩阵乘法的耗时占比如何变化？其他内核呢？

提交内容：用 1-2 句话回答。

(e) 比较模型自注意力层在前向传播中 softmax 运算与矩阵乘法运算的运行时间。运行时间的差异与 FLOP 数量的差异相比如何？

提交内容：用 1-2 句话回答。

2.1.5 混合精度

到目前为止，我们一直使用 FP32 精度：所有模型参数和激活值的数据类型都是 `torch.float32`。不过，现代 NVIDIA GPU 包含专用 GPU 核心（Tensor Core），用于加速低精度矩阵乘法。例如，NVIDIA B200 的规格表给出的 FP32 最大吞吐量为 80 TFLOPS，而使用 FP16（半精度浮点数）或 BF16（`bfloat16`）时，最大吞吐量显著提高，达到惊人的 2500 TFLOPS。因此，使用较低精度的数据类型应当有助于加快训练和推理。

然而，直接将模型转换为低精度格式可能会降低模型精度。例如，实践中的许多梯度值小到无法用 FP16 表示，因此直接用 FP16 训练时会变成零。为了解决这个问题，使用 FP16 训练时通常会采用损失缩放：将损失乘以一个缩放因子，以增大梯度的幅值，防止它们下溢为零。此外，FP16 的动态范围比 FP32 小，可能发生溢出，表现为损失变成 NaN。完全使用 `bfloat16` 训练通常更稳定，因为 BF16 与 FP32 具有相同的动态范围；但相比 FP32，它仍可能影响模型的最终表现。

为了获得低精度数据类型带来的加速，通常会采用混合精度训练。在 PyTorch 中，这通过 `torch.autocast` 上下文管理器实现。某些操作（例如矩阵乘法）使用低精度数据类型执行，而需要 FP32 完整动态范围的其他操作（例如累加与归约）则保持原样。例如，下面的代码会自动识别前向传播中哪些操作应使用低精度，并将这些操作转换到指定的数据类型：

```
model : torch.nn.Module = ... # 例如，你的 Transformer 模型
dtype : torch.dtype = ... # 例如，torch.bfloat16
x : torch.Tensor = ... # 输入数据
with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

如上所述，即使待累加的张量本身已转换为较低精度，通常也应该用较高精度进行累加。下面的练习将帮助你建立这方面的直觉。

题目 (`mixed_precision_accumulation`)：混合精度累加（1 分）

运行下面的代码，并评价结果的准确性。

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)
s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
```

```

s += torch.tensor(0.01, dtype=torch.float16)
print(s)
s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01, dtype=torch.float16)
    s += x.type(torch.float32)
print(s)

```

提交内容：用 2–3 句话回答。

接下来，我们先在一个玩具模型上应用混合精度以建立直觉，然后将它应用于基准测试脚本。

题目 (benchmarking_mixed_precision)：混合精度基准测试 (2 分)

(a) 考虑以下模型：

```

class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x

```

假设我们在 GPU 上训练这个模型，模型参数最初为 FP32，并希望使用 FP16 自动转换混合精度。以下各项的数据类型分别是什么？

- autocast 上下文中的模型参数；
- 第一个前馈层 (ToyModel.fc1) 的输出；
- 层归一化 (ToyModel.ln) 的输出；
- 模型预测的 logits；
- 损失；
- 模型梯度。

提交内容：列出上述每一项的数据类型。

(b) 你应该已经看到，FP16 混合精度自动转换对层归一化与前馈层采取了不同的处理方式。层归一化中的哪些部分对混合精度敏感？如果使用 BF16 而不是 FP16，是否仍需要对层归一化做不同处理？为什么？

提交内容：用 2–3 句话回答。

(c) 修改基准测试脚本，使其可以选择使用 BF16 混合精度运行模型。对第 2.1.2 节的每种语言模型规模，分别测量使用和不使用混合精度时前向、反向传播的时间。比较全精度与混合精度的结果，并评论随模型规模变化出现的趋势。nullcontext 这个不执行任何操作的上下文管理器可能会有用。

提交内容：用 2–3 句话回答，包含计时结果和评论。

2.1.6 显存分析

到目前为止，我们一直关注计算性能。现在将注意力转向显存——它是语言模型训练和推理中的另一项主要资源。PyTorch 也提供了强大的显存分析器，可以跟踪随时间发生的内存分配。

要使用显存分析器，可以按如下方式修改基准测试脚本：

```
... # 基准测试脚本的预热阶段
# 开始记录显存历史。
torch.cuda.memory._record_memory_history(max_entries=1000000)
... # 基准测试脚本中需要分析的部分
# 保存 pickle 文件，供 PyTorch 在线工具加载。
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")
# 停止记录历史。
torch.cuda.memory._record_memory_history(enabled=None)
```

这会输出一个 memory_snapshot.pickle 文件，可以将其加载到在线工具 https://pytorch.org/memory_viz 中。该工具既能显示整体显存使用时间线，也能显示每一次分配的大小及其来源代码的调用栈。使用时，在浏览器中打开上述链接，然后将 pickle 文件拖放到页面上即可。

接下来，你将使用 PyTorch 性能分析器分析模型的显存使用情况。

题目 (memory_profiling)：显存分析 (4 分)

对表 1 中的 xl 模型，在上下文长度为 128 和 2048 时，分析包含前向传播、反向传播和优化器更新的完整训练步骤。

(a) 在性能分析脚本中添加选项，让模型通过显存分析器运行。

复用之前的基础设施可能会很有帮助，例如启用混合精度、加载指定模型规模等。随后运行脚本，获得 xl 模型仅执行推理（只有前向传播）以及执行完整训练步骤时的显存分析结果。显存时间线是什么样的？能否根据观察到的峰值，判断当前正在执行哪个阶段？

提交内容：来自 memory_viz 工具的两张 xl 模型 “Active memory timeline”（活动显存时间线）图：一张对应前向传播，另一张对应完整训练步骤（前向、反向传播以及优化器更新）；另附 2–3 句话的回答。

(b) 对每个上下文长度，前向传播时的峰值显存是多少？完整训练步骤的峰值显存又是多少？

提交内容：一张表格，每个上下文长度对应两个数值。

(c) 找出 xl 模型使用混合精度时，前向传播和完整训练步骤的峰值显存。混合精度是否显著影响显存使用量？

提交内容：用 2-3 句话回答。

(d) 考虑 xl 模型。在给定的参考超参数下，Transformer 残差流中的一个激活张量，如果使用单精度，其大小是多少？请以 MiB 为单位给出结果，即将字节数除以 1024^2 。

提交内容：用 1-2 句话回答，并给出推导。

(e) 现在仔细观察 https://pytorch.org/memory_viz 中 xl 模型前向传播显存快照的 “Active Memory Timeline”。降低 “Detail”（细节）级别时，工具会隐藏相应比例的较小分配，例如设为 10% 时，只显示最大的 10% 分配。所显示的最大分配大小是多少？查看调用栈，你能否判断这些分配来自哪里？

提交内容：用 1-2 句话回答。

(f) Nsight Systems 也有用于显存分析的选项。你可以将这些选项与前面使用的 Nsight 选项结合，了解模型生命周期各阶段发生了哪些分配。使用 PyTorch 提供的 NVTX 标签，确定模型中一个 TransformerBlock 为反向传播保存了多少显存；这些张量通常称为 residuals。记录贡献最大的 5 个操作，以及它们各自占总显存的百分比。

在反向传播期间，这些张量都会被释放，但与此同时还会产生新的梯度张量。根据分析结果中前向传播分配的显存量，以及反向传播中每个 TransformerBlock 对显存使用量的改变，计算一个 TransformerBlock 产生的梯度张量占用多少显存。结果符合预期吗？

提交内容：Nsight Systems 截图，以及 1-2 段回答。

3 单 GPU 显存

本次作业后续将探索如何把张量分片到多个 GPU，但有些技巧即使在单 GPU 训练中也能使用。其中最常见的是梯度检查点（gradient checkpointing），也称为激活检查点（activation checkpointing）。

3.1 Autograd 保存张量

回忆一下：为了对模型执行反向传播，我们需要保存前向传播生成的激活。某些操作显然需要这样做，但默认情况下，实际保存张量的操作可能比你想象的多得多。为反向传播保存的张量称为 “residuals”，或简称为 “saved tensors”（保存张量）。

让我们了解网络中保存了什么。先从看似简单的 RMSNorm 函数入手；为简便起见，使用纯 FP32。我们在 autograd 保存或取回张量时添加一些钩子。

```

import torch
from torch import nn
x = torch.randn((4, 512, 2560), requires_grad=True)
class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps
    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x
def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t
def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f>Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t
ln = RMSNorm(x.shape[-1])
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()

```

输出表明，保存的张量数量多得令人担忧，其中几个甚至与完整激活张量一样大！

```

$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>

```

```

Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None

```

3.1.1 算子融合

在这种情况下，显然是使用的操作划分得过于细碎。我们希望用一个操作接收 RMSNorm 权重和激活并输出结果，同时希望它在反向传播中也作为一个整体操作。这是进行内核融合的动机之一。RMSNorm 的行为相对规整，因此我们甚至可以使用 `torch.compile` 自动融合它。

```

...
ln = torch.compile(RMSNorm(x.shape[-1]))
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()

```

新的输出明显改善：

```

Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None

```

现在只需要为反向传播保存一个完整大小的激活张量，即 RMSNorm 函数的输入。还要注意，加载张量的顺序不再是保存顺序的逆序，而且每个保存张量都不再具有 `grad_fn` 依赖；PyTorch 正在将整个 RMSNorm 视为单个函数。

3.2 激活检查点

融合无疑很有用，但它能节约的显存仍然有限。例如，我们融合一个 xl 规模的 TransformerBlock：

```

import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock
# 此模型的 num_layers 为 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
    positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))
# 尽可能在 torch.compile 允许的范围内进行融合
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)

```

```

...
# 现在记录所保存的字节数
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter): # 跳过参数记录, 避免重复计数
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t
...
# 执行前向传播, 保存反向传播所需张量
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)
print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes /
(1024**2):.2f} MiB")

```

脚本会显示, 为反向传播保存了多少显存:

```

...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB

```

每层 3.6 GiB。如果对所有层都这样做, 仅仅是为反向传播保存的激活就会达到 114 GiB! 注意力运算保存的张量中存在相当多的浪费, 我们将在第 4 节解决这一问题。但即使修复了这部分问题, 显存使用量仍会随批大小、序列长度和嵌入维度线性增长。

3.2.1 重计算

不必一直保留生成的每个张量: 我们可以只定期保存结果作为检查点, 再重新计算检查点之间的值。PyTorch 提供了一个接口, 可以很方便地实现这一点。`torch.utils.checkpoint.checkpoint` 接收一个函数及传给该函数的参数, 然后以下述方式改变该函数的行为:

1. 在前向传播中:
 - (a) 保存函数的输入值。
 - (b) 抑制前向传播中的张量保存。
2. 在反向传播中:
 - (a) 首先执行重计算: 根据先前保存的输入重新执行前向传播, 并保存反向传播所需的值。
 - (b) 执行反向传播, 随后即可释放所有张量。

对于依次运行 4 个 Transformer block 的简单情形, 可以看到显存按预期累加:

```

...
def four_blocks(x):
    x = block(x)

```

```

    x = block(x)
    x = block(x)
    x = block(x)
    return x
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)
print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes /
(1024**2):.2f} MiB")

```

Total size of saved tensors in four TransformerBlocks: 14605.25 MiB

不过，我们可以采用如下梯度检查点方法：

```

from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x
def four_blocks_checkpoint(x):
    # checkpoint 在反向传播之前丢弃所有保存张量
    # 反向传播到达使用检查点的块时，
    # 会重新执行前向传播，生成需要保存的张量，
    # 然后完成正常的反向传播。
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)
print(f"Total size of saved tensors in four TransformerBlocks with checkpointing:
{total_size_bytes / (1024**2):.2f} MiB")

```

```

Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB

```

请记住，这并没有消除显存需求，而是将显存使用分成了两类：一类是在每次 checkpoint 调用入口处保存、用于后续重计算的长期存储，即检查点本身；另一类是在检查点块内部重计算时临时产生、用于完成该块反向传播的短期存储。由于主要关注的是峰值显存，我们希望平衡保存检查点的显存成本与完整生成一个块所需保存张量的显存成本。使用的检查点越多，单个块内需要同时生成的显存量越小，但检查点本身占用的显存越多。

在上面的例子中，重计算生成的保存张量显然占据了主要显存，一部分原因是检查点本身很小。因此，将两者的平衡向检查点存储倾斜会更有利，也就是说，应减小一个检查点块所覆盖的范围。

检查点块变大或变小，不会改变重计算的计算成本。不过，通过使用递归检查点，也就是在 checkpoint 调用内部再嵌套 checkpoint 调用，我们可以用更多计算来进一步降低显存需求。

题目 (gradient_checkpointing): 显存最优的梯度检查点 (4 分)

考虑一个由 N 个相同块顺序堆叠而成的 Transformer。如果不使用检查点，全部 N 个块的保存张量会同时驻留，峰值激活显存为 $O(N)$ 。我们可以自由地用 checkpoint 包装前向传播的任意部分，包括将 checkpoint 调用彼此嵌套。

(a) 忽略计算成本，什么检查点策略能够最小化峰值激活显存？描述你如何安排 checkpoint 调用（可以给出代码草图），并给出该策略的渐近峰值激活显存和计算量，作为 N 的函数。假设单个块保存的张量占用远大于每个检查点的管理开销。

提交内容：用 3–5 句话描述策略及其渐近峰值显存，并给出简短的代码草图。

(b) 考虑上面批大小为 4、序列长度为 2048 的 xl 模型配置。如果时间/计算预算只允许执行一轮重计算，也就是说不能嵌套 checkpoint 调用，那么降低峰值显存的最佳检查点策略是什么？分析运行的峰值显存以验证假设，并比较紧邻的更小和更大的检查点块大小，确认判断。

提交内容：用 3–5 句话说明推理过程，并给出该策略测得的峰值显存。

4 GPU 内核

4.1 使用 FlashAttention-2 优化注意力

4.1.1 PyTorch 注意力基准测试

你的性能分析很可能表明，注意力层在显存和计算方面都存在优化机会。从总体上看，注意力运算由一次矩阵乘法、softmax，以及另一次矩阵乘法组成：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right)V \quad (1)$$

朴素注意力实现需要为每个批次/注意力头元素保存形状为 $\text{seq_len} \times \text{seq_len}$ 的注意力分数矩阵。序列较长时，这些矩阵可能变得非常大，导致长输入或长输出任务发生显存不足错误。我们将根据 FlashAttention-2 论文实现一个注意力内核：它按分块计算注意力，避免显式生成完整的 $\text{seq_len} \times \text{seq_len}$ 注意力分数矩阵，从而支持远长于原先的序列。

题目 (pytorch_attention): PyTorch 注意力基准测试 (2 分)

(a) 在不同规模下对你的注意力实现进行基准测试。编写脚本完成以下工作:

- (i) 将批大小固定为 8, 不使用多头注意力, 即去掉注意力头维度。
- (ii) 遍历头嵌入维度 d_{model} 的取值 [16, 32, 64, 128] 与序列长度 [256, 1024, 4096, 8192, 16384] 的笛卡尔积。
- (iii) 为对应尺寸创建随机输入 Q, K, V 。
- (iv) 使用这些输入, 测量注意力运算 100 次前向传播的时间。
- (v) 测量反向传播开始前已使用的显存, 并测量 100 次反向传播的时间。
- (vi) 确保进行了预热, 并在每次前向/反向传播之后调用 `torch.cuda.synchronize()`。

根据 GPU 的不同, 部分配置预计会出现显存不足。报告这些配置的耗时或显存不足错误。在何种规模下开始发生显存不足? 从导致显存不足的最小配置中选一个, 核算注意力的显存使用量; 你可以使用作业 1 中 Transformer 显存使用的公式。为反向传播保存的显存如何随序列长度变化? 你会如何消除这部分显存开销?

提交内容: 一张耗时表、显存使用量的计算过程, 以及 1-2 段回答。

4.2 即时编译注意力的基准测试

从 2.0 版本起, PyTorch 还提供了强大的即时编译器, 能够自动尝试对 PyTorch 函数应用多种优化。入门介绍见: https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html。特别是, 它会动态分析计算图, 并尝试自动生成融合后的 Triton 内核。PyTorch 编译器的接口很简单。例如, 若要将其应用于模型的单个层, 可以使用:

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

现在, `compiled_layer` 在功能上与 `layer` 完全一致, 包括前向传播和反向传播。也可以使用 `torch.compile(model)` 编译整个 PyTorch 模型, 甚至编译一个调用 PyTorch 操作的 Python 函数。

题目 (torch_compile): Torch Compile (2 分)

- (a) 扩展注意力基准测试脚本, 加入你实现的 PyTorch 注意力的编译版本, 并在与上面 `pytorch_attention` 题目相同的配置下, 将其性能与未编译版本比较。

提交内容：一张表格，比较编译后的注意力模块与前面 `pytorch_attention` 题目中未编译版本的前向、反向传播耗时。

(b) 现在，在端到端基准测试脚本中编译整个 Transformer 模型。前向传播的性能如何变化？将前向传播、反向传播和优化器更新合在一起时，性能又如何？

提交内容：一张比较普通 Transformer 与编译后模型的表格。

根据已经观察到的序列长度相关扩展行为，要处理长序列，我们需要显著的改进。即便使用 `torch.compile`，当前实现在长序列下仍有很差的内存访问模式。为此，我们将用 Triton 实现 FlashAttention-2，以便更精细地控制如何访问内存，以及何时执行哪些计算。

4.2.1 示例——加权求和

为了介绍使用 Triton 及其与 PyTorch 互操作所需的知识，我们将讲解一个“加权求和”运算的示例内核。更多 Triton 入门资源可参考[官方教程](#)。需要指出，这些教程没有使用较新且方便的块指针（block pointer）抽象；我们将在下文介绍它。

给定输入矩阵 X ，我们将其各元素乘以按列对应的权重向量 w ，再对每一行求和，得到 X 与 w 的矩阵-向量乘积。首先讲解这个运算的前向传播，再编写其反向传播的 Triton 内核。

前向传播

内核的前向传播就是下面这个使用广播的内积运算：

```
def weighted_sum(x, weight):  
    # 假设 x 为形状 [..., D] 的 n 维张量，weight 为形状 [D] 的一维张量  
    return (weight * x).sum(axis=-1)
```

编写 Triton 内核时，我们让每个程序实例（它们可能并行运行）计算 x 中一个行分块的加权和，并将相应的标量结果写入输出张量。在 Triton 中，一个程序实例就是一组运行相同程序的线程构成的线程块，这些线程块可以在 GPU 上并行运行。我们不再把张量作为参数传入，而是传入指向张量首元素的指针，以及用于说明如何沿各轴移动的步幅（stride）。

我们可以利用步幅加载当前实例所负责求和的 x 行分块，并通过程序 ID 分配工作，即实例 i 处理 x 的第 i 个行分块。在这个简单情形中，Triton 与 PyTorch 前向传播的主要区别，是需要进行指针运算并显式加载/存储数据。我们将使用 `tl.make_block_ptr` 的块指针抽象，大幅简化指针运算；不过，这也意味着需要先做一些准备工作来构造块指针。

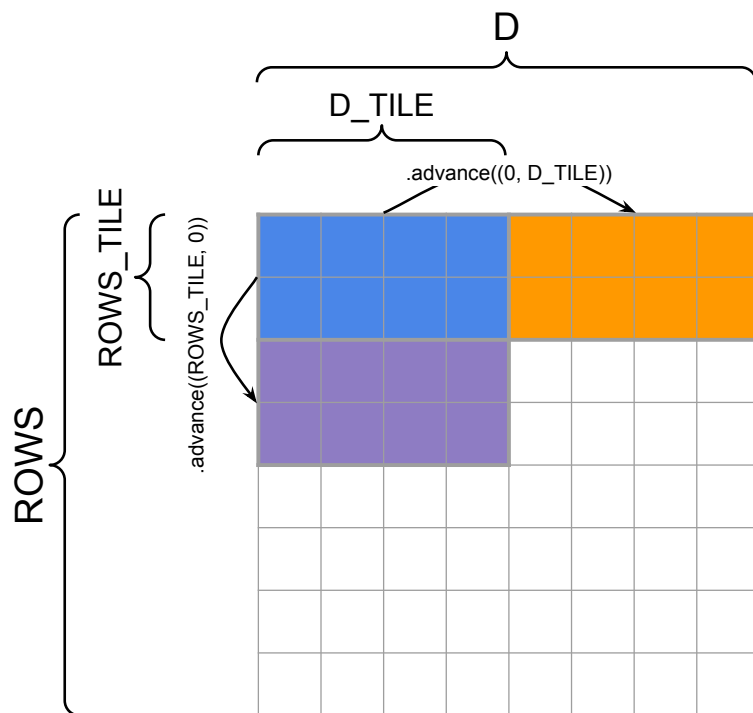


图 2: 加权求和内核示例（第 4.2.1 节）中的分块及块指针推进。

图 2 展示了分块以及如何推进块指针。上面的加权求和函数可以写成：

```
import triton
import triton.language as tl
@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr, # 输入指针
    output_ptr, # 输出指针
    x_stride_row, x_stride_dim, # 步幅说明沿张量每个轴移动一个元素的方式
    weight_stride_dim, # 通常为 1
    output_stride_row, # 通常为 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr, # 编译时必须已知分块形状
):
    # 每个实例计算 x 的一个行分块的加权和。
    # 通过 tl.program_id 确定当前运行于哪个线程块
    row_tile_idx = tl.program_id(0)
    # 块指针用于选择内存中的一个 N 维区域，
    # 并移动所选区域。
    # 块指针必须知道：
    # - 指向张量首元素的指针
    # - 张量的整体形状，以处理越界访问
    # - 各维度的步幅，以正确使用内存布局
    # - 起始分块的 N 维坐标，即 offsets
    # - 每次加载或存储的分块形状
    # - 内存中各维度从主轴到次轴的顺序
    #   (= np.argsort(strides))，用于优化；
    #   >=Hopper 的 TMA 支持需要此信息
```

```

x_block_ptr = tl.make_block_ptr(
    x_ptr,
    shape=(NUM_ROWS, D,),
    strides=(x_stride_row, x_stride_dim),
    offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
    block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
    order=(1, 0),
)
weight_block_ptr = tl.make_block_ptr(
    weight_ptr,
    shape=(D,),
    strides=(weight_stride_dim,),
    offsets=(0,),
    block_shape=(D_TILE_SIZE,),
    order=(0,),
)
output_block_ptr = tl.make_block_ptr(
    output_ptr,
    shape=(NUM_ROWS,),
    strides=(output_stride_row,),
    offsets=(row_tile_idx * ROWS_TILE_SIZE,),
    block_shape=(ROWS_TILE_SIZE,),
    order=(0,),
)
# 初始化用于写入的缓冲区
output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)
for i in range(tl.cdiv(D, D_TILE_SIZE)):
    # 加载当前块指针所指数据
    # ROWS_TILE_SIZE 可能无法整除 NUM_ROWS, D_TILE_SIZE 可能无法整除 D,
    # 因此两个维度都需要边界检查
    row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE,
    D_TILE_SIZE)
    weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero") # (D_TILE_SIZE,)
    # 计算每一行的加权和。
    output += tl.sum(row * weight[None, :], axis=1)
    # 将指针移到下一个分块。
    # 这里给出的是 (行, 列) 坐标增量
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE)) # 沿最后一个维度移动 D_TILE_SIZE
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,)) # 移动 D_TILE_SIZE
# 将输出写入输出块指针所指位置, 每行一个标量。
# ROWS_TILE_SIZE 可能无法整除 NUM_ROWS, 因此需要边界检查
tl.store(output_block_ptr, output, boundary_check=(0,))

```

现在将该内核包装成一个 PyTorch Autograd 函数, 使其能够与 PyTorch 互操作: 输入为 Tensor, 输出为 Tensor, 随后还能在反向传播中配合 autograd 引擎工作。

```

class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # 缓存 x 和 weight, 供反向传播使用;
        # 反向传播中只接收相对于输出张量的梯度,
        # 需要计算相对于 x 和 weight 的梯度。
        D, output_dims = x.shape[-1], x.shape[:-1]
        # 将输入张量重塑为二维
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")
        ctx.save_for_backward(x, weight)

```

```

assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"
ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16 # 沿嵌入维度循环约 16 次
ctx.ROWS_TILE_SIZE = 16 # 每个线程一次处理 16 个批元素
ctx.input_shape = input_shape
# 初始化空的结果张量；注意其中元素不一定为 0!
y = torch.empty(output_dims, device=x.device)
# 在一维网格中启动 n 个内核实例。
n_rows = y.numel()
weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
    x, weight,
    y,
    x.stride(0), x.stride(1),
    weight.stride(0),
    y.stride(0),
    NUM_ROWS=n_rows, D=D,
    ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
)
return y.view(input_shape[:-1])

```

注意，在调用 Triton 内核 `weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]` 时，我们通过传入元组 `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)`，定义了线程块的“启动网格”（launch grid）。随后可在内核中用 `tl.program_id(0)` 获取线程块索引。

反向传播

由于定义了自己的内核，我们也需要编写自己的反向传播函数。

在前向传播中，我们得到层的输入，需要计算输出。回忆反向传播：给定目标函数相对于输出的梯度，需要计算它相对于每个输入的梯度。在本例中，运算的输入是矩阵 $x : \mathbb{R}^{n \times h}$ 和权重向量 $w : \mathbb{R}^h$ 。为简洁起见，将运算记为 $f(x, w)$ ，其值域为 $\mathbb{R}^n$ 。假设已知损失 $\mathcal{L}$ 相对于该层输出的梯度 $\nabla_{f(x, w)} \mathcal{L}$ ，则应用多元链式法则可得相对于 x 和 w 的梯度：

$$(\nabla_x \mathcal{L})_{ij} = \sum_{k=1}^n \frac{\partial f(x, w)_k}{\partial x_{ij}} (\nabla_{f(x, w)} \mathcal{L})_k = w_j \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (2)$$

$$(\nabla_w \mathcal{L})_j = \sum_{i=1}^n \frac{\partial f(x, w)_i}{\partial w_j} (\nabla_{f(x, w)} \mathcal{L})_i = \sum_{i=1}^n x_{ij} \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (3)$$

这给出了计算反向传播的简单公式。对于 x 的反向传播，应用式 (2)，取 w 与 $\nabla_{f(x, w)}$ 的外积。对于 w 的反向传播，即 $(\nabla_w \mathcal{L})_j$ ，必须将传入的梯度与对应的输出行相乘。

反向传播内核首先定义所有块指针，然后计算 $\nabla_x \mathcal{L}$ ：

```

@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr, # 输入
    grad_output_ptr, # 输入梯度
    grad_x_ptr, partial_grad_weight_ptr, # 输出梯度
    stride_xr, stride_xd,

```

```

stride_wd,
stride_gr,
stride_gxr, stride_gxd,
stride_gwb, stride_gwd,
NUM_ROWS, D,
ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)
    # 输入
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )
    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )
    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )
    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )
    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero") #
        (ROWS_TILE_SIZE,)
        # 通过外积计算 grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero") # (D_TILE_SIZE,)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))
        # 尽可能归约更多行, 计算 grad_weight 结果
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE,
        D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,)) # 第 0
        维永远不会越界
        # 沿 D 维将指针移到下一个分块
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))

```

计算梯度 ∇_x 很简单，我们将结果写入输出张量的对应分块。不过，计算 ∇_w 更有挑战。每个内核实例负责 x 的一个行分块，但现在需要跨 x 的各行求和。我们不直接在反向传播中完成这个求和，而是假设 `partial_grad_weight_ptr` 指向一个 `n_row_tiles  $\times$  H` 的矩阵，其中第一维只在 x 的单个行分块内部做了归约。写入该张量前，先在当前行分块内部归约。在内核之外，再使用 `torch.sum` 对各行分块的结果求和，完成 ∇_w 的归约。¹

因此，`autograd.Function` 的最后一部分相对简单：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (前面已定义)
    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE # 这些值不必与前向传播相同
        n_rows, D = x.shape
        # 策略是让每个线程块先写入部分结果缓冲区，
        # 再归约该缓冲区，得到最终梯度。
        partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.device,
                                           dtype=x.dtype)
        grad_x = torch.empty_like(x)
        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

最后，可以得到一个使用方式很像 `torch.nn.functional` 中函数的接口：

```
f_weightedsum = WeightedSumFunc.apply
```

现在，对两个 PyTorch 张量 x 和 w 调用 `f_weightedsum`，会得到如下张量：

```
tensor([ 90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192],
        device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

注意张量附带的 `grad_fn`：它说明，当该张量出现在计算图中时，PyTorch 知道在反向传播中该调用什么。至此，我们完成了加权求和运算的 Triton 实现。

4.2.2 FlashAttention-2 前向传播

你将依据 FlashAttention-2 [T. Dao, 2023]，用显著改进的 Triton 实现替换原来的 PyTorch 注意力实现。FlashAttention-2 使用一些技巧进行分块前向计算，以获得高效的内存访问模式，并避免在全局内存中生成完整注意力矩阵。

¹ 当然，也可以为此编写自己的内核。

开始本节之前，我们强烈建议至少阅读原始 FlashAttention 论文 [T. Dao et al., 2022]。它有助于理解 FlashAttention 实现高效注意力的核心技巧：跨分块以在线方式计算 softmax；这种方法由 [M. Milakov et al., 2018] 提出。我们也建议阅读 H. He [4]，以便进一步直观理解 GPU 实际如何执行 PyTorch 代码。

理解普通注意力的低效之处

回忆一下，注意力的前向传播（暂时忽略掩码）可写成：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d} \quad (4)$$

$$P_{ij} = \text{softmax}_j(\mathbf{S})_{ij} \quad (5)$$

$$\mathbf{O} = \mathbf{P}\mathbf{V} \quad (6)$$

标准反向传播为：

$$d\mathbf{V} = \mathbf{P}^\top d\mathbf{O} \quad (7)$$

$$d\mathbf{P} = d\mathbf{O}\mathbf{V}^\top \quad (8)$$

$$d\mathbf{S}_i = d\text{softmax}(d\mathbf{P}_i) = (\text{diag}(\mathbf{P}_i) - \mathbf{P}_i\mathbf{P}_i^\top) d\mathbf{P}_i \quad (9)$$

$$d\mathbf{Q} = d\mathbf{S}\mathbf{K} / \sqrt{d} \quad (10)$$

$$d\mathbf{K} = d\mathbf{S}^\top \mathbf{Q} / \sqrt{d} \quad (11)$$

可以看到，反向传播依赖前向传播中一些非常大的激活。例如，式 (7) 中计算 $d\mathbf{V}$ 需要 $\mathbf{P}$ ，也就是形状为 $(\text{batch_size}, \text{n_heads}, \text{seq_len}, \text{seq_len})$ 的注意力分数。该激活矩阵的大小随序列长度二次增长，这解释了先前对长序列注意力进行基准测试时遇到的显存问题。在普通注意力的前向和反向传播中，将 $\mathbf{P}$ 及其他大型激活在片上 SRAM 与 GPU HBM 之间传输，会带来显著的内存 IO 开销。标准实现中会发生多次这样的传输：例如，标准反向传播实现在式 (7) 和式 (9) 的计算中都会从 HBM 读取 $\mathbf{P}$ 。

FlashAttention 的主要目标是避免在 HBM 中读写注意力矩阵，从而降低 IO 和峰值显存成本。我们使用三项技术来实现：分块、重计算和算子融合。

分块

为了避免在 HBM 中读写注意力矩阵，我们在无法同时访问全部输入的情况下完成 softmax 归约。具体而言，重新组织注意力计算，将输入拆成多个分块，并对输入分块进行多轮处理，从而逐步完成 softmax 归约。

重计算

我们避免将形状为 $(\text{batch_size}, \text{n_heads}, \text{seq_len}, \text{seq_len})$ 的大型中间注意力矩阵保存在 HBM 中，而是保存某些“激活检查点”，在反向传播时重新计算一部分前向传播，以获得计算梯度所需的其他激活。FlashAttention-2 还会保存注意力分数的 logsumexp 值 L ，用来简化反向传播计算。 L 的表达式为：

$$L_i = \log \left(\sum_j \exp(S_{ij}) \right) \quad (12)$$

在最终内核中，我们将以在线方式计算这个值，但最终结果应当相同。结合分块与重计算之后，内存 IO 和峰值使用量不再依赖 sequence_length^2 ，因此可以使用更长的序列。

算子融合

最后，我们把所有操作放在单个内核中执行，从而避免对注意力矩阵及其他中间激活重复进行内存 IO；这称为算子融合或内核融合。我们会编写一个单独的 Triton 前向内核，在 HBM 与 SRAM 之间只进行有限的数据传输，就完成注意力涉及的全部操作。重计算是支持算子融合的部分原因，因为它让我们能够避免通常为每个中间激活写入 HBM 所付出的内存 IO 成本。

关于这些技术的更多直观解释，可参阅 FlashAttention 论文 [T. Dao et al., 2022; T. Dao, 2023]。

带重计算的反向传播

利用 L ，我们可以进行适当的重计算并高效完成反向传播。在反向传播开始前，先在全局内存中预计算 $D = \text{rowsum}(\mathbf{O} \circ \mathbf{dO})$ ，其中 $\circ$ 表示逐元素乘法。它等于 $\text{rowsum}(\mathbf{P} \circ \mathbf{dP})$ ，因为

$$\mathbf{P}\mathbf{dP}^\top = \mathbf{P}(\mathbf{dO}\mathbf{V}^\top)^\top = (\mathbf{P}\mathbf{V})\mathbf{dO}^\top = \mathbf{O}\mathbf{dO}^\top,$$

且对任意矩阵 $\mathbf{A}, \mathbf{B}$ ，都有 $\text{rowsum}(\mathbf{A} \circ \mathbf{B}) = \text{diag}(\mathbf{A}\mathbf{B}^\top)$ 。有了向量 L 和 D ，反向传播就不再需要 softmax。完整计算如下：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d} \quad (13)$$

$$P_{ij} = \exp(S_{ij} - L_i) \quad (14)$$

$$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \quad (15)$$

$$\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \quad (16)$$

$$\mathbf{dS}_{ij} = P_{ij}(\mathbf{dP}_{ij} - D_i) \quad (17)$$

$$\mathbf{dQ} = \mathbf{dS}\mathbf{K} / \sqrt{d} \quad (18)$$

$$\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q} / \sqrt{d} \quad (19)$$

可以看到，这一系列操作不要求在前向传播时把注意力分数 $\mathbf{P}$ 保存在 HBM 中；我们通过式 (13) 和式 (14)，从激活 $\mathbf{Q}, \mathbf{K}, L$ 重新计算它们。

FlashAttention 前向传播的细节

了解 FlashAttention-2 所用技术的总体思路之后，我们将深入讨论你需要实现的 FA2 前向内核。为了避免在 HBM 中读写注意力矩阵，我们希望采用分块，即独立计算输出的每个分块。这要求能够计算 P 的分块，最好同时沿查询与键两个维度分块。

然而，对 S 应用 softmax 时，需要归约 S 的整行才能计算 softmax 分母，因此不能直接分块计算 P 。FlashAttention-2 用在线 softmax 解决这个问题。下文用下标 i 表示当前查询分块，用上标 (j) 表示当前键分块。查询维度的分块大小为 B_q ，键维度的分块大小为 B_k ，不沿隐藏维度 d 分块。

我们还维护逐行更新的运行值 $m_i^{(j)} \in \mathbb{R}^{B_q}$ 和 $l_i^{(j)} \in \mathbb{R}^{B_q}$ 。逐行的 $m_i^{(j)}$ 是运行最大值，用于以数值稳定的方式计算 softmax；回忆作业 1 中 softmax 实现采用的技巧。每当读入一个新的 S 的逐行分块，即 j 增大时，我们就更新 $m_i^{(j)}$ 。

利用运行最大值，可以计算未归一化的 softmax 值（分子）： $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_{ij} - m_i^{(j)})$ 。 $l_i^{(j)}$ 是 softmax 分母的运行代理值，并利用未归一化的 softmax 值更新为

$$l_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) \cdot l_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}).$$

最终写出输出时，需要使用 $l_i^{(T_k)}$ 完成归一化；它是处理完所有键分块后 $l_i^{(j)}$ 的最终值。算法 1 展示了 GPU 上的前向传播。

算法 1: FlashAttention-2 前向传播

- 1 **输入:** $\mathbf{Q} \in \mathbb{R}^{N_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N_k \times d}$, 分块大小 B_q, B_k 。
 - 2 将 $\mathbf{Q}$ 拆成 $T_q = \lceil N_q/B_q \rceil$ 个大小为 $B_q \times d$ 的分块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_q}$ 。
 - 3 将 $\mathbf{K}, \mathbf{V}$ 拆成 $T_k = \lceil N_k/B_k \rceil$ 个大小为 $B_k \times d$ 的分块 $\mathbf{K}^{(1)}, \dots, \mathbf{K}^{(T_k)}$ 和 $\mathbf{V}^{(1)}, \dots, \mathbf{V}^{(T_k)}$ 。
 - 4 **对于** $i = 1, \dots, T_q$:
 - 5 从全局内存加载 $\mathbf{Q}_i$ 。
 - 6 初始化 $\mathbf{O}_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}$, $l_i^{(0)} = 0 \in \mathbb{R}^{B_q}$, $m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$ 。
 - 7 **对于** $j = 1, \dots, T_k$:
 - 8 从全局内存加载 $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$ 。
 - 9 计算 softmax 前的注意力分数分块 $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i(\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 。
 - 10 计算 $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_q}$ 。
 - 11 计算 $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_q \times B_k}$ 。
 - 12 计算 $l_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)})l_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_q}$ 。
 - 13 计算 $\mathbf{O}_i^{(j)} = \text{diag}(\exp(m_i^{(j-1)} - m_i^{(j)}))\mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)}\mathbf{V}^{(j)}$ 。
 - 14 **结束内层循环。**
 - 15 计算 $\mathbf{O}_i = \text{diag}(l_i^{(T_k)})^{-1}\mathbf{O}_i^{(T_k)}$ 。
 - 16 计算 $L_i = m_i^{(T_k)} + \log(l_i^{(T_k)})$ 。
 - 17 将 $\mathbf{O}_i$ 写入全局内存，作为 $\mathbf{O}$ 的第 i 个分块。
 - 18 将 L_i 写入全局内存，作为 L 的第 i 个分块。
 - 19 **结束外层循环。**
 - 20 返回输出 $\mathbf{O}$ 和 logsumexp 值 L 。
-

在开始用 Triton 实现前向传播之前，先汇总一些编写 Triton 内核的通用技巧。

Triton 提示与技巧

- 可以使用 `tl.device_print` 在 Triton 中打印信息进行调试：https://triton-lang.org/main/python-api/generated/triton.language.device_print.html。设置 `TRITON_INTERPRET=1` 可以在 CPU 上运行 Triton 解释器，不过我们发现它存在一些问题。
- 定义块指针时，确保偏移量正确，而且块偏移已经乘以相应的分块大小。
- 线程块的启动网格通过以下方式设置：

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

这段调用位于 `torch.autograd.Function` 子类的方法中，正如加权求和示例所示。

- 使用 `tl.dot` 进行矩阵乘法。
- 推进块指针时，使用 `*_block_ptr = *_block_ptr.advance(...)`。

题目 (flash_forward): FlashAttention-2 前向传播 (15 分)

(a) 编写一个纯 PyTorch (不使用 Triton) 的 `autograd.Function`，实现 FlashAttention-2 前向传播。它会比普通 PyTorch 实现慢得多，但有助于调试 Triton 内核。

实现应接收输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 和一个 `is_causal` 标志，生成输出 $\mathbf{O}$ 和 `logsumexp` 值 L 。本小题可以忽略 `is_causal`。`autograd.Function` 的 `forward` 应当保存 L, Q, K, V, O 以供反向传播使用，并返回 O 。记住，`autograd.Function` 的 `forward` 方法总是把上下文作为第一个参数。任何 `autograd.Function` 类都需要实现 `backward` 方法，不过目前可以让它直接抛出 `NotImplementedError`。如果需要对照结果，可以在 PyTorch 中实现式 (4)–(6) 和式 (12)，再比较输出。

接口为 `def forward(ctx, Q, K, V, is_causal=False)`。分块大小由你决定，但必须至少为 16×16 。测试维度始终至少为 16，且是 2 的幂，因此不用担心越界访问。

提交内容： 一个在前向传播中实现 FlashAttention-2 的 `torch.autograd.Function` 子类。为测试代码，实现 `[adapters.get_flashattention_autograd_function_pytorch]`；随后运行 `uv run pytest -k test_flash_forward_pass_pytorch`，确保实现通过测试。

(b) 按照算法 1 编写 FlashAttention-2 前向传播的 Triton 内核。然后再编写一个 `torch.autograd.Function` 子类，在前向传播中调用这个融合内核，而不是用 PyTorch 计算结果。以下提示与本题有关：

- 调试时，建议把每个 Triton 操作的结果，与 (a) 中编写的分块 PyTorch 实现比较。
- 启动网格应设为 $(T_q, \text{batch_size})$ 。这意味着每个 Triton 程序实例只加载某一个批次索引对应的元素，而且只读写 $\mathbf{Q}, \mathbf{O}, L$ 的一个查询分块。
- 内核只应包含一个循环，遍历键分块 $1 \leq j \leq T_k$ 。
- 在循环末尾推进块指针。
- 使用下面的函数声明。根据我们提供的块指针，你应当能够推断其余指针的设置方式：

```
@triton.jit
def flash_fwd_kernel(
```

```

Q_ptr, K_ptr, V_ptr,
Q_ptr, L_ptr,
stride_qb, stride_qq, stride_qd,
stride_kb, stride_kk, stride_kd,
stride_vb, stride_vk, stride_vd,
stride_ob, stride_oq, stride_od,
stride_lb, stride_lq,
N_QUERIES, N_KEYS,
scale,
D: tl.constexpr,
Q_TILE_SIZE: tl.constexpr,
K_TILE_SIZE: tl.constexpr,
):
    # 程序索引
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)
    # 用对应的批次索引,
    # 乘以各张量的批次步幅, 为每个指针增加偏移
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )
    ...

```

其中 $scale$ 为 $1/\sqrt{d}$, Q_TILE_SIZE 和 K_TILE_SIZE 分别对应 B_q 和 B_k 。之后可以调优这些值。

以下补充建议有助于避免精度问题：

- 片上缓冲区 (O_i, l, m) 的数据类型应为 `tl.float32`。若要向输出缓冲区累加, 使用 `acc` 参数: `acc = tl.dot(..., acc=acc)`。
- 将 $\tilde{P}_i^{(j)}$ 与 $V^{(j)}$ 相乘之前, 先转换为 $V^{(j)}$ 的数据类型; 把 O_i 写回全局内存之前, 也要转换为恰当的数据类型。类型转换使用 `tensor.to`; 张量的数据类型可用 `tensor.dtype` 获取, 块指针/指针所指元素的数据类型可用 `*_block_ptr.type.element_ty` 获取。

提交内容: 一个使用你编写的 Triton 内核, 在前向传播中实现 FlashAttention-2 的 `torch.autograd.Function` 子类。实现 `[adapters.get_flash_autograd_function_triton]`, 随后运行 `uv run pytest -k test_flash_forward_pass_triton`, 确保通过测试。

(c) 给 `autograd.Function` 实现增加一个用于因果掩码的标志, 作为最后一个参数。它应为布尔值, 设为 `True` 时, 通过索引比较启用因果掩码。Triton 内核应增加相应参数 `is_causal: tl.constexpr`; 该类型注解是必需的。在 Triton 中构造适当的查询与键索引向量, 并比较它们, 形成大小为 $B_q \times B_k$ 的方形掩码。对于被屏蔽的元素, 向注意力分数矩阵 $S_i^{(j)}$ 中的相应元素加上常量 `-1e6`。确保通过 `ctx.is_causal = is_causal` 保存掩码标志, 供反向传播使用。

提交内容: 为 `torch.autograd.Function` 子类增加一个标志, 使其能够通过 Triton 内核实现带因果掩码的 FlashAttention-2 前向传播。该标志必须是可选参数, 默认值为 `False`, 以便之前的测试仍然通过。

使用重计算实现反向传播

注意，与式 (7)–(11) 的标准反向传播不同，式 (13)–(19) 所示的反向传播可以用重计算避开 softmax 运算。因此，我们可以通过一个简单内核完成反向传播，而不需要任何在线计算技巧。本部分可以在常规 PyTorch 函数（不使用 Triton）上调用 `torch.compile` 来实现 backward。

题目 (flash_backward): FlashAttention-2 反向传播 (5 分)

使用 PyTorch（不使用 Triton）和 `torch.compile`，实现 FlashAttention-2 的 `autograd.Function` 的反向传播。实现应接收 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, d\mathbf{O}, L$ 张量，并返回 $d\mathbf{Q}, d\mathbf{K}, d\mathbf{V}$ 。记得计算并使用向量 D 。可以按照式 (13)–(19) 的计算过程实现。

提交内容：运行 `uv run pytest -k test_flash_backward` 来测试实现。

现在，将这个部分使用 Triton 的 FlashAttention-2 实现，与常规注意力的 PyTorch 实现进行性能比较。

题目 (flash_benchmarking): FlashAttention-2 基准测试 (5 分)

(a) 使用 `triton.testing.do_bench` 编写基准测试脚本，比较部分使用 Triton 的 FlashAttention-2 实现与常规 PyTorch 实现（不使用 FlashAttention）的前向、反向传播性能。

具体而言，需要提供一张表格，列出 Triton 与 PyTorch 两种实现的前向传播、反向传播，以及端到端前向加反向传播的延迟。计时开始前，随机生成所有需要的输入，并在单张 B200 上运行基准测试。始终使用批大小 1，并启用因果掩码。遍历以下组合的笛卡尔积：从 128 到 65536 的各个 2 的幂作为序列长度，从 16 到 128 的各个 2 的幂作为嵌入维度，以及 `torch.bfloat16` 和 `torch.float32` 两种精度。你很可能需要根据输入尺寸调整分块大小。

提交内容：一张结果表，使用上述设置比较你的 FlashAttention-2 实现与 PyTorch 实现，报告前向传播、反向传播及端到端延迟。

4.2.3 可选：Triton 反向传播

如果希望获得更多 Triton 实践经验，和/或提交速度较快的排行榜实现，我们在下面提供了可以用 Triton 实现的分块 FlashAttention-2 反向传播。算法 2 展示了应当如何在 Triton 中实现它。这里的关键技巧是计算两次 $\mathbf{P}$ ：一次用于 $d\mathbf{Q}$ ，另一次用于 $d\mathbf{K}$ 和 $d\mathbf{V}$ 。这样可以跳过线程块之间的同步，避免缓慢的原子操作。

算法 2: 分块 FlashAttention-2 反向传播

- 1 **输入:** $\mathbf{Q}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N_k \times d}$, $L \in \mathbb{R}^{N_q}$, 分块大小 B_q, B_k 。
 - 2 计算 $D = \text{rowsum}(\mathbf{O} \circ \mathbf{dO}) \in \mathbb{R}^{N_q}$ 。
 - 3 将 $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ 拆成 $T_q = \lceil N_q/B_q \rceil$ 个分块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_q}$, $\mathbf{O}_1, \dots, \mathbf{O}_{T_q}$, $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_q}$; 每个大小为 $B_q \times d$ 。
 - 4 将 $\mathbf{K}, \mathbf{V}$ 拆成 $T_k = \lceil N_k/B_k \rceil$ 个分块 $\mathbf{K}^{(1)}, \dots, \mathbf{K}^{(T_k)}$ 和 $\mathbf{V}^{(1)}, \dots, \mathbf{V}^{(T_k)}$; 每个大小为 $B_k \times d$ 。
 - 5 将 L, D 拆成 T_q 个分块 $L_1, \dots, L_{T_q}$ 和 $D_1, \dots, D_{T_q}$; 每个大小为 B_q 。
 - 6 **对于** $j = 1, \dots, T_k$:
 - 7 从全局内存加载 $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$ 。
 - 8 初始化 $\mathbf{dK}_0^{(j)} = \mathbf{dV}_0^{(j)} = \mathbf{0} \in \mathbb{R}^{B_k \times d}$ 。
 - 9 **对于** $i = 1, \dots, T_q$:
 - 10 从全局内存加载 $\mathbf{Q}_i, \mathbf{dO}_i$ 。
 - 11 计算注意力分数分块 $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i (\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 。
 - 12 计算注意力概率 $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_q \times B_k}$ 。
 - 13 计算 $\mathbf{dV}_i^{(j)} = \mathbf{dV}_{i-1}^{(j)} + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_k \times d}$ 。
 - 14 计算 $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i (\mathbf{V}^{(j)})^\top \in \mathbb{R}^{B_q \times B_k}$ 。
 - 15 计算 $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_q \times B_k}$ 。
 - 16 计算 $\mathbf{dK}_i^{(j)} = \mathbf{dK}_{i-1}^{(j)} + (\mathbf{dS}_i^{(j)})^\top \mathbf{Q}_i / \sqrt{d} \in \mathbb{R}^{B_k \times d}$ 。
 - 17 **结束内层循环。**
 - 18 将 $\mathbf{dK}_{T_q}^{(j)}$ 与 $\mathbf{dV}_{T_q}^{(j)}$ 写入全局内存, 作为 $\mathbf{dK}$ 与 $\mathbf{dV}$ 的第 j 个分块。
 - 19 **结束外层循环。**
 - 20 **对于** $i = 1, \dots, T_q$:
 - 21 从全局内存加载 $\mathbf{Q}_i, \mathbf{dO}_i$ 。
 - 22 初始化 $\mathbf{dQ}_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}$ 。
 - 23 **对于** $j = 1, \dots, T_k$:
 - 24 从全局内存加载 $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$ 。
 - 25 计算注意力分数分块 $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i (\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 。
 - 26 计算注意力概率 $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_q \times B_k}$ 。
 - 27 计算 $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i (\mathbf{V}^{(j)})^\top \in \mathbb{R}^{B_q \times B_k}$ 。
 - 28 计算 $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_q \times B_k}$ 。
 - 29 计算 $\mathbf{dQ}_i^{(j)} = \mathbf{dQ}_i^{(j-1)} + \mathbf{dS}_i^{(j)} \mathbf{K}^{(j)} / \sqrt{d} \in \mathbb{R}^{B_q \times d}$ 。
 - 30 **结束内层循环。**
 - 31 将 $\mathbf{dQ}_i^{(T_k)}$ 写入全局内存, 作为 $\mathbf{dQ}$ 的第 i 个分块。
 - 32 **结束外层循环。**
 - 33 返回 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 。
-

5 分布式数据并行训练

在作业的下一部分, 我们将探索使用多个 GPU 训练语言模型的方法, 重点是数据并行。首先介绍 PyTorch 中的分布式通信, 然后研究一种朴素的分布式数据并行训练实现, 接着实现并测试多种提高通信效率的方法。

5.1 PyTorch 中的单节点分布式通信

先来看一个简单的 PyTorch 分布式应用，目标是生成四个随机整数张量，并计算它们的和。

下面的分布式实现会启动四个工作进程，每个进程生成一个随机整数张量。为了跨进程对这些张量求和，我们调用 all-reduce 集合通信操作，用全归约结果（即总和）替换每个进程中原来的数据张量。

来看代码：

```
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp
def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)
def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")
if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上述脚本后，会得到下面的输出。与预期一致，每个工作进程最初持有不同的数据张量。all-reduce 对全部工作进程的张量求和之后，各工作进程中的 data 都被原地修改，保存全归约后的结果。²

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

现在更仔细地回顾上面的脚本。mp.spawn 启动 nprocs 个进程，用给定的 args 运行 fn。此外，函数以 fn(rank, *args) 的形式调用，其中 rank 是工作进程索引，取值从 0 到 nprocs-1。因此，distributed_demo 必须将这个整数 rank 作为第一个位置参数。

² 多次运行该脚本时，你会发现打印顺序不确定。由于应用以分布式方式运行，我们无法控制命令执行的精确先后顺序；唯一的保证是，all-reduce 完成后，各进程持有逐比特相同的结果张量。

此外，我们还传入 `world_size`，它表示工作进程总数。

每个工作进程都属于一个通过 `dist.init_process_group` 初始化的进程组。进程组表示多个通过共享主节点进行协调和通信的工作进程。主节点由 IP 地址与端口指定，rank 为 0 的进程运行在主节点上。all-reduce 等集合通信操作作用于进程组中的每个进程。

本例使用 "gloo" 后端初始化进程组，不过也有其他后端。特别是，"nccl" 后端会使用 NVIDIA 的 NCCL 集合通信库，对 CUDA 张量通常有更好的性能。但是，NCCL 只能用于带 GPU 的机器，而 Gloo 可以在仅有 CPU 的机器上运行。分布式 GPU 训练应始终使用 NCCL；只有本地开发且没有 GPU 时才使用 Gloo。本例使用 Gloo，是为了允许在仅有 CPU 的机器上本地执行和开发。

运行多 GPU 任务时，要确保不同 rank 使用不同 GPU。一种方法是在 `setup` 函数中调用 `torch.cuda.set_device(rank)`，这样 `tensor.to("cuda")` 就会自动将张量移到指定设备。另一种方法是显式创建每个 rank 的设备字符串，例如 `device = f"cuda:{rank}"`，然后在任何数据搬运操作中将该字符串用作目标设备，例如 `tensor.to(f"cuda:{rank}")`。

术语

在作业后续部分以及网上的其他资料中，你可能会在 PyTorch 分布式通信的语境里遇到以下术语。虽然本作业重点是单节点、多进程分布式训练，这些术语对理解一般分布式训练也很有帮助。可参考图 3 的示意图。

node（节点）

网络上的一台机器。

worker（工作者）

参与分布式训练的一个程序实例。本作业中，每个 worker 只有一个进程，因此 worker、process 和 worker process 可以互换使用。但实际中，一个 worker 可能使用多个进程，例如用于加载训练数据，所以这些术语并不总是等价。

world size（全局工作者数）

一个进程组中的工作者总数。

global rank（全局编号）

进程组中用于唯一标识工作者的整数 ID，范围为 0 到 `world_size-1`。例如，world size 为 2 时，一个进程的 global rank 为 0（主进程），另一个为 1。

local world size（本地工作者数）

应用跨多个节点运行时，某个节点上本地运行的工作者数量。例如，在 2 个节点上分别启动 4 个工作者，world size 为 8，local world size 为 4。注意，单节点运行时，工作者的 local world size 等于全局 world size。

local rank（本地编号）

用于唯一标识某台机器上本地工作者索引的整数 ID，范围为 0 到 `local_world_size-1`。例如，在 2 个节点上分别启动 4 个进程，每个节点的工作者的 local rank 都为 0、1、2、3。注意，运行单节点多进程分布式应用时，一个进程的 local rank 等于其 global rank。

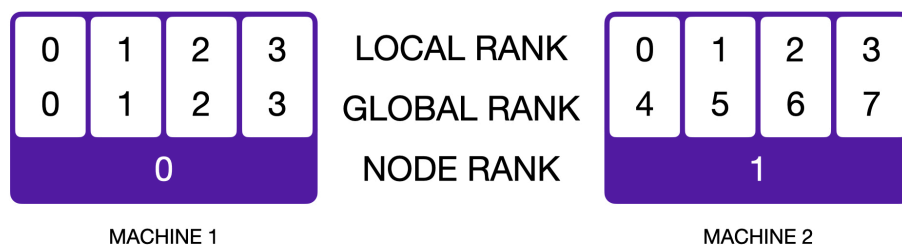


图 3: 一个在 2 个节点上运行、world size 为 8 的分布式应用示意图。每个工作进程由 global rank (0-7) 和 local rank (0-3) 标识。
图片来源: https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

5.1.1 分布式应用基准测试的最佳实践

在本部分中, 你将通过分布式应用的基准测试, 更好地理解通信开销。以下是一些最佳实践:

- 尽可能在同一台机器上运行基准测试, 以便进行受控比较。
- 在对目标操作计时之前, 先执行若干预热步骤。对于 NCCL 通信调用, 这尤其重要。通常预热 5 次迭代就足够。
- 在 GPU 上进行基准测试时, 调用 `torch.cuda.synchronize()`, 等待 CUDA 操作完成。即便通信操作使用 `async_op=False`, 也仍然需要同步: 该调用在操作被排入 GPU 队列时返回, 而不是在通信实际完成后才返回。³
- 不同 rank 的计时结果可能略有不同, 因此通常会汇总多个 rank 的测量值, 以改进估计。可以使用 `all-gather` 集合通信, 尤其是 `dist.all_gather_object`, 来收集各 rank 的结果。
- 一般而言, 先在本地 CPU 上用 Gloo 调试, 再根据题目要求, 在 GPU 上用 NCCL 进行基准测试。

切换后端应当只需要修改 `init_process_group` 调用, 以及张量设备转换代码。

题目 (`distributed_communication_single_node`): 分布式通信 (单节点) (5 分)

编写脚本, 在单节点、多进程设置下对 `all-reduce` 操作的运行时间进行基准测试。前面的示例代码可以作为起点。尝试改变以下设置:

all-reduce 数据大小: 使用 `float32` 张量, 大小分别为 1MB、10MB、100MB、1GB。

GPU / 进程数量: 2、4 或 6。

资源需求: 最多 6 个 GPU。每次基准测试应少于 5 分钟。

提交内容: 比较各种设置的图和/或表, 以及 2-3 句话的评论, 说明结果, 并讨论不同因素之间如何相互影响。

³ 更多细节见 <https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386>。

5.2 分布式数据并行训练的朴素实现

了解了在 PyTorch 中编写分布式应用的基础后，我们来构建一个最简的分布式数据并行（Distributed Data Parallel, DDP）训练实现。

数据并行会把一个批次拆分到多个设备（例如 GPU）上，使我们能够使用单个设备容纳不下的大批次进行训练。例如，4 个设备中的每个设备最多处理批大小 32，数据并行就能支持有效批大小 128。

下面是朴素分布式数据并行训练的步骤。首先，每个设备构建一个随机初始化的模型。使用 broadcast 集合通信操作，把 rank 0 的模型参数发送到所有其他 rank。训练开始时，每个设备持有完全相同的模型参数与优化器状态副本，例如 Adam 中累积的梯度统计量。

1. 给定包含 n 个样本的批次，将其分片，每个设备收到互不重叠的 n/d 个样本，其中 d 为数据并行设备数。 d 应能整除 n ，否则某些 rank 的工作量会大于其他 rank，整个步骤将受最慢者限制。
2. 每个设备用本地模型参数，在自己的 n/d 个样本上执行前向传播，再执行反向传播计算梯度。此时，每个设备持有的梯度仅由它收到的 n/d 个样本计算而来。
3. 使用 all-reduce 集合通信，对不同设备的梯度取平均，使每个设备持有在全部 n 个样本上平均后的梯度。
4. 每个设备执行优化器更新，更新自己的参数副本。从优化器的角度看，它只是在优化一个本地模型。由于各设备从相同的模型和优化器状态开始，并在每次迭代中使用相同的平均梯度，因此所有设备的参数和优化器状态会保持同步。至此，完成一次训练迭代，之后可重复上述过程。

题目 (naive_ddp): 朴素 DDP (5 分)

提交内容：实现一种朴素的分布式数据并行训练，在反向传播后分别对各参数梯度执行 all-reduce。为测试实现，请实现 `[adapters.get_ddp]`，以及可选的 `[adapters.ddp_on_after_backward]`，再运行 `uv run pytest tests/test_ddp.py`。

题目 (naive_ddp_benchmarking): 朴素 DDP 基准测试 (3 分)

在这种朴素 DDP 实现中，每次反向传播后，分别对每个参数梯度跨 rank 执行 all-reduce。为了更好地理解数据并行训练的开销，编写一个脚本，对之前实现的语言模型采用此 DDP 实现训练时的性能进行基准测试。测量每个训练步骤的总时间，以及用于梯度通信的时间比例。在单节点设置（1 个节点 $\times$ 2 个 GPU）下，对第 2.1.2 节的 xl 模型规模收集测量结果。

提交内容：描述基准测试设置，并给出每种设置下每次训练迭代的耗时及梯度通信耗时。

5.3 改进最简 DDP 实现

第 5.2 节的最简 DDP 实现有两个主要局限：

1. 对每个参数张量单独执行一次 all-reduce。每次通信调用都会引入开销，因此批量执行通信可能有助于减小这些开销。
2. 等待整个反向传播结束后才通信梯度。然而，反向传播是逐步计算的。一个参数梯度就绪时，就可以立即开始通信，而不必等待其他参数的梯度。这让梯度通信与反向传播计算重叠，从而降低分布式数据并行训练的开销。

本部分将依次解决这些局限，并测量改进对训练速度的影响。

5.3.1 减少通信调用次数

与其为每个参数张量发起一次通信，不如看看批量执行 all-reduce 能否改善性能。具体而言，将需要 all-reduce 的梯度拼接成单个张量，然后跨全部 rank 对合并后的梯度执行 all-reduce。torch._utils._flatten_dense_tensors 和 torch._utils._unflatten_dense_tensors 可能会有帮助。

题目 (minimal_ddp_flat_benchmarking)：扁平化梯度的最简 DDP 基准测试 (2 分)

修改最简 DDP 实现，把所有参数的梯度展平到一个张量中进行通信。在之前使用的条件下，即 1 个节点 × 2 个 GPU、第 2.1.2 节中的 xl 模型规模，将其与对每个参数张量分别执行 all-reduce 的最简 DDP 实现进行性能比较。

提交内容：采用单次批量 all-reduce 的分布式数据并行训练中，每次训练迭代的耗时和梯度通信耗时；再用 1–2 句话比较批量通信与逐个梯度通信的结果。

5.3.2 让计算与单个参数梯度的通信重叠

批量通信可能降低大量小型 all-reduce 操作的调用开销，但全部通信时间仍然直接计入额外开销。为解决这个问题，我们可以利用以下观察：反向传播从损失开始向输入推进，逐层计算梯度。因此，参数梯度一旦就绪，就可以立即 all-reduce，让反向传播计算与梯度通信重叠，减少数据并行训练的开销。

我们首先实现并测试一个分布式数据并行包装器：在反向传播中，各参数张量的梯度一旦就绪，就异步执行 all-reduce。以下提示可能有帮助：

反向传播钩子：要在参数梯度通过反向传播累积完成后，自动对该参数调用一个函数，可以使用 register_post_accumulate_grad_hook。⁴

⁴ 更多信息与用法示例见 https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html。

异步通信：PyTorch 的全部集合通信操作都支持同步执行（`async_op=False`）和异步执行（`async_op=True`）。同步调用会阻塞，直到集合通信操作被排入 GPU 队列。这并不意味着 CUDA 操作已完成，因为 CUDA 操作是异步的。不过，后续使用其输出的函数调用会按预期工作。⁵ 相比之下，异步调用会返回一个分布式请求句柄；因此，函数返回时，集合通信操作不保证已进入 GPU 队列，更不保证已经完成。可以对返回的通信句柄调用 `handle.wait()`，等待操作被排入 GPU 队列，从而使后续操作能够使用输出。

例如，下面两段示例分别通过同步调用和异步调用，对张量列表中的每个张量执行 all-reduce：

```
tensors = [torch.rand(5) for _ in range(10)]
# 同步：阻塞，直到操作被排入 GPU 队列。
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)
# 异步：每次调用后立即返回，
# 最后再等待结果。
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)
# ...
# 此处可执行不依赖 all_reduce 结果的其他命令
# ...
# 确保 all_reduce 调用已进入队列，
# 从而可以继续排入依赖
# all_reduce 输出的其他操作。
for handle in handles:
    handle.wait()
handles.clear()
```

题目 (`ddp_overlap_individual_parameters`)：与单个参数通信重叠的 DDP（5 分）

实现一个 Python 类，处理分布式数据并行训练。该类应包装任意 PyTorch `nn.Module`，负责在训练前广播权重，使所有 rank 具有相同的初始参数，并发起用于梯度平均的通信调用。建议采用以下公开接口：

```
def __init__(self, module: torch.nn.Module):
```

给定一个已实例化、需要并行化的 PyTorch `nn.Module`，构建一个负责跨 rank 梯度同步的 DDP 容器。

```
def forward(self, *inputs, **kwargs):
```

使用给定的位置参数和关键字参数，调用被包装模块的 `forward()` 方法。

⁵ 在高级场景中，如果使用多个 CUDA stream，可能需要显式的跨流同步，确保输出已准备好供后续操作使用。参见 <https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams>。

```
def finish_gradient_synchronization(self):
```

调用时，等待异步通信调用在 GPU 上完成。

使用该类进行分布式训练时，先传入要包装的模块，然后在 `optimizer.step()` 之前调用 `finish_gradient_synchronization()`，确保依赖梯度的优化器更新操作可以安全地排入执行队列：

```
model = ToyModel().to(device)
ddp_model = DDP(model)
for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

提交内容：实现一个处理分布式数据并行训练的容器类，使梯度通信与反向传播计算重叠。为测试 DDP 类，先实现适配器 `[adapters.get_ddp]` 和 `[adapters.ddp_on_after_backward]`；后者是可选的，根据实现可能不需要。

随后运行 `uv run pytest tests/test_ddp.py` 执行测试。建议多次运行测试，例如 5 次，以确保能够稳定通过。

题目 (`ddp_overlap_individual_parameters_benchmarking`)：单个参数通信重叠的 DDP 基准测试（1 分）

(a) 对反向传播计算与单个参数梯度通信重叠的 DDP 实现进行性能测试。在相同配置下，即 1 个节点、2 个 GPU、第 2.1.2 节的 `xl` 模型规模，与之前研究的两种实现比较：对每个参数张量单独 `all-reduce` 的最简 DDP，以及对全部参数张量的拼接结果执行一次 `all-reduce` 的最简 DDP。

提交内容：测量反向传播与单个参数梯度通信重叠时，每次训练迭代的耗时，并用 1-2 句话比较结果。

(b) 在基准测试代码中加入 `Nsight` 性能分析，使用 1 个节点、2 个 GPU、`xl` 模型规模，比较初始 DDP 实现与这种重叠实现。直观比较两份执行轨迹，并提供性能分析截图，说明一种实现让计算与通信重叠，而另一种没有。

提交内容：两张截图：一张来自初始 DDP 实现，另一张来自计算与通信重叠的 DDP 实现。截图应直观展示通信是否与反向传播重叠。

6 优化器状态分片

分布式数据并行训练概念简单，通常也很有效，但它要求每个 rank 各自持有一份模型参数和优化器状态。这样的冗余会带来显著的显存成本。例如，AdamW 优化器为每个参数维护两个浮点数，因此其显存消耗是模型权重的两倍。S. Rajbhandari et al. [5] 提出了几种减少数据并行训练中这种冗余的方法：将 (1) 优化器状态、(2) 梯度和 (3) 参数分配到各 rank，并在需要时在工作者之间通信。

本部分将实现一个简化的优化器状态分片方案，降低每个 rank 的显存消耗。每个 rank 上的优化器实例不再保留全部参数的优化器状态，而只处理一个参数子集，大约为总参数的 $1 / \text{world_size}$ 。当各 rank 的优化器执行更新步骤时，只更新自己分片中的模型参数。之后，每个 rank 将更新后的参数广播给其他 rank，确保每次优化器更新之后模型参数仍然同步。

题目 (optimizer_state_sharding): 优化器状态分片 (15 分)

实现一个处理优化器状态分片的 Python 类。该类应包装任意传入的 PyTorch `optim.Optimizer`，并负责在每次优化器更新之后同步更新后的参数。建议采用以下公开接口：

```
def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any):
```

初始化状态分片优化器。params 是需要优化的参数集合，或者参数组；参数组允许用户对模型不同部分使用不同超参数，例如学习率。这些参数将被分片到全部 rank。optimizer_cls 指定被包装优化器的类型，例如 `optim.AdamW`。其余关键字参数都转交给 optimizer_cls 的构造函数。确保在此方法中调用父类 `torch.optim.Optimizer` 的构造函数。

```
def step(self, closure, **kwargs):
```

使用提供的 closure 和关键字参数，调用被包装优化器的 `step()` 方法。更新参数后，与其他 rank 同步。

```
def add_param_group(self, param_group: dict[str, Any]):
```

将一个参数组添加到分片优化器中。该方法在分片优化器构建期间由父类构造函数调用，也可能在训练中调用，例如逐步解冻模型层时。因此，该方法应负责在各 rank 之间分配模型参数。

提交内容：实现一个处理优化器状态分片的容器类。为测试分片优化器，先实现适配器 `[adapters.get_sharded_optimizer]`，再运行 `uv run pytest tests/test_sharded_optimizer.py`。建议多次运行测试，例如 5 次，以确保能够稳定通过。

完成优化器状态分片之后，让我们分析它对训练峰值显存的影响，以及它引入的运行时间开销。

题目 (optimizer_state_sharding_accounting): 优化器状态分片的开销核算 (5 分)

(a) 编写脚本，分析使用与不使用优化器状态分片训练语言模型时的峰值显存。采用标准配置，即 1 个节点、2 个 GPU、xl 模型规模，报告模型初始化后、优化器更新之前，以及优化器更新之后的峰值显存。结果符合预期吗？分解每种设置下的显存使用，例如参数占多少、优化器状态占多少等。

提交内容：用 2-3 句话回答，包含峰值显存结果，以及显存在不同模型和优化器组件之间的分配明细。

(b) 我们实现的优化器状态分片如何影响训练速度？在标准配置（1 个节点、2 个 GPU、xl 模型规模）下，分别测量使用与不使用优化器状态分片时，每次迭代所需的时间。

提交内容：用 2-3 句话回答，包含计时结果。

(c) 我们的优化器状态分片方法与 ZeRO stage 1 有何不同？后者在 S. Rajbhandari、J. Rasley、O. Ruwase 和 Y. He [5] 中称为 ZeRO-DP P_{os} 。

提交内容：用 2-3 句话总结差异，特别是与显存和通信量有关的差异。

7 全分片数据并行

借助优化器状态分片和数据并行，我们已经能够沿数据并行轴拆分优化器状态与激活。然而，模型权重仍然是重复的：每个 GPU 都保存了所有权重的完整副本。

可以把数据并行（DP）轴变成全分片数据并行（FSDP）轴来解决这个问题。使用 FSDP 时，每个 GPU 只保存每个权重张量中属于自己的切片，但为了准备前向或反向传播，需要通过 all-gather 从其他 GPU 拉取切片，重建完整权重张量。

为了避免 GPU 计算等待通信完成，大多数 FSDP 实现会提前调度层的 all-gather，使相应权重在真正需要之前就准备好，避免通信阻塞计算。这样，权重分片通信就不再位于关键路径上；只要通信能够跟上计算且调度得当，就不会带来额外时间成本。

有些层的显存占用和计算量很小，传输的延迟开销并不值得。这些层应被标记为不参与 FSDP 分片。在我们的架构中，这主要指归一化层，因此剩下需要分片的是嵌入层和各个线性层。

尽管主权重必须以 FP32 存储，因为被反复累加的值对精度敏感，但参与计算的权重不需要使用 FP32。在混合精度下，使用前总会将权重转换成低精度计算类型，因此不妨在通信之前就转换，以节省带宽。

题目 (fsdp): 全分片数据并行 (15 分)

实现一个用于全分片数据并行训练的 Python 类。该类应包装任意 PyTorch `nn.Module`，即你的完整模型，并为其中的任何 `Linear` 或 `Embedding` 层安装钩子或进行包装。建议采用以下公开接口：

```
def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None):
```

给定一个已实例化、需要并行化的 PyTorch `nn.Module`，构建一个负责权重 all-gather 和梯度 reduce-scatter 的 FSDP 模块。确保钩子或模块包装器及时收集权重，供前向传播使用。为限制显存，只有在当前层之前的第二个层完成前向传播后，才开始收集当前层的权重。在反向传播中，钩子或模块包装器同样应通过 all-gather 准备计算所需权重。梯度就绪后，应通过 reduce-scatter 分发到相应 rank。使用完毕后，务必释放收集到的权重。若提供了 `compute_dtype`，则在通信或计算使用前，将权重转换为该类型；同时保持主权重及优化器更新为 FP32。

```
def forward(self, *inputs, **kwargs):
```

使用给定的位置参数和关键字参数，调用被包装模块的 `forward()` 方法。

```
def finish_gradient_synchronization(self):
```

调用时，等待异步通信调用在 GPU 上完成。

提交内容：实现一个处理全分片数据并行训练的容器类。该容器的每个分片应兼容作业 1 的标准 AdamW 实现。为测试 FSDP 实现，请实现适配器 `[adapters.get_fsdp]`，并运行 `uv run pytest tests/test_fsdp.py`。建议多次运行测试，例如 5 次，以发现潜在的竞态条件。

题目 (`fsdp_accounting`)：FSDP 开销核算（5 分）

(a) 根据第 6 节的分析，你预计实现 FSDP 后能够减少多少峰值显存？计算时，可以忽略为了将权重 all-gather 到各 GPU 而预先分配的缓冲区大小。

提交内容：用 2–3 句话回答，给出你的结论。

(b) 在两个 GPU 上分析 `xl` 模型，重点观察权重 all-gather。通信是否能及时完成，赶上前向传播？

提交内容：用 2–3 句话回答，包含计时结果，并附 Nsight 截图作为支持。

8 并行策略分析

还可以沿其他轴并行化训练。常见策略包括：

- **数据并行 (DP)：**将数据批次拆分到多个设备，每个设备计算自身批次的梯度，随后跨设备取梯度平均值。
- **全分片数据并行 (FSDP)：**在数据并行基础上，进一步将优化器状态、梯度和权重拆分到多个设备，以降低显存占用。设备需要在前向、反向传播期间，从其他设备收集权重分片。

- **张量并行 (TP)**: 沿输入或输出维度对权重矩阵分片。各设备计算自己分片对应的激活, 然后跨设备对激活归约或收集。
- **流水线并行 (PP)**: 按层将模型拆成多个阶段, 每个阶段在不同设备上运行。
- **专家并行 (EP)**: 将混合专家模型中的专家拆分到不同设备, 各设备计算自己负责的专家的输出。

本节将在简化设置下做一些基本的数学计算, 用来选择并行策略并决定如何组合它们。首先关注 DP、FSDP、TP 及其组合。我们的方法是计算每种策略的通信成本, 并将其与计算成本比较, 从而判断在通信成为瓶颈之前能够扩展到多少设备。

关于 TPU / GPU 拓扑和并行策略的更详细讨论, [TPU Scaling Book](#) (J. Austin et al. [6]) 是很好的资源。更详细的流水线并行讨论, 可参阅《[超大规模训练手册](#)》附录 (H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7])。这两本书的其他部分也包含许多有用信息。

8.1 通信原语

首先理解通信原语。在简化设置中, 假设有 N 个设备, 编号为 $0, \dots, N-1$, 每对设备之间都有链路连接。还假设每个设备的出口 (即向外发送) 带宽为 W , 也就是说, 每个设备能以每秒 W 字节的速度向另一个设备发送数据。可以实现 gather 和 reduce?

all-gather 的一种常见实现是环形 *all-gather*。回忆 all-gather: 每个设备 i 最初持有大小为 S/N 的一个分块 x_i , 最后得到完整的 $x = [x_0, \dots, x_{N-1}]$, 总大小为 S 字节。环形 all-gather 将设备排成一个环。每一步, 每个设备向右侧相邻设备发送当前分块, 并保存从左侧设备收到的分块。不断重复, 每个设备将刚收到的分块继续向右传递, 同时从左边接收新的分块。经过 $N-1$ 步, 每个设备都得到完整张量。

在这个理想化设置中, 每一步各设备同时通过出口带宽 W 发送一个大小为 S/N 的分块, 一共有 $N-1$ 步, 因此环形 all-gather 耗时为 $\frac{N-1}{N} \frac{S}{W}$ 秒。

接下来分析环形 *reduce-scatter*。在 reduce-scatter 中, 每个设备 i 最初持有大小为 S 的完整张量 $x^{(i)}$ 。我们希望计算归约 $y = \sum_{i=0}^{N-1} x^{(i)}$, 但最终每个设备 i 只保留大小为 S/N 的分块 y_i 。与环形 all-gather 一样, 先将设备排成一个环。各设备将自身张量 $x^{(i)}$ 拆成 N 个分块 $[x_0^{(i)}, \dots, x_{N-1}^{(i)}]$, 每块大小为 S/N 。随后像环形 all-gather 那样传递分块, 但每次转发之前, 设备先将自身的贡献加到该分块中; 分块保存的是部分和。具体而言:

对于步骤 $t = 1, \dots, N-1$, 设备 i 执行以下操作:

- 若 $t = 1$, 初始化 $y \leftarrow x^{(i)}$, 保存目前的部分和。
- 向设备 $(i+1) \bmod N$ 发送分块 $y_{(i-t) \bmod N}$ 。
- 从设备 $(i-1) \bmod N$ 接收分块 $z_{(i-t-1) \bmod N}$ 。
- 更新本地部分和: $y_{(i-t-1) \bmod N} \leftarrow y_{(i-t-1) \bmod N} + z_{(i-t-1) \bmod N}$ 。

经过 $N-1$ 步, 设备 i 得到分块 y_i 的完整和。因此, 环形 reduce-scatter 耗时为 $\frac{N-1}{N} \frac{S}{W}$ 秒, 与环形 all-gather 相同。

最后实现环形 *all-reduce*。在 *all-reduce* 中，每个设备 i 最初持有大小为 S 的完整张量 $x^{(i)}$ ，最终得到归约 $y = \sum_{i=0}^{N-1} x^{(i)}$ 。我们先执行环形 *reduce-scatter*，再执行环形 *all-gather* 来实现 *all-reduce*，因此环形 *all-reduce* 耗时为 $2 \frac{N-1}{N} \frac{S}{W}$ 秒。

题目 (`alternate_ring_all_reduce`): 另一种环形 *all-reduce* (1 分)

不采用“环形 *reduce-scatter* 后接环形 *all-gather*”的实现方式，而使用下面的算法：

对于步骤 $t = 1, \dots, N-1$ ，设备 i 执行以下操作：

- 若 $t = 1$ ，初始化 $y \leftarrow x^{(i)}$ ，保存目前的部分和。
- 向设备 $(i+1) \bmod N$ 发送 $x^{((i-t+1) \bmod N)}$ 。
- 从设备 $(i-1) \bmod N$ 接收 $x^{((i-t) \bmod N)}$ 。
- 更新本地部分和： $y \leftarrow y + x^{((i-t) \bmod N)}$ 。

在与前面相同的设置下，即每个设备出口带宽为 W 、每个 $x^{(i)}$ 大小为 S ，该算法需要多长时间？

提交内容：用 S, N, W 表示答案，并用一句话说明理由。

8.2 数据并行分析

有了通信原语，便可以分析并行策略。我们将分析一个 FFN 层的并行化。回忆一下，给定输入 $\mathbf{x}$ ，前向传播为：

$$\mathbf{x}_1 = \mathbf{x} \mathbf{W}_1 \quad (20)$$

$$\mathbf{x}_2 = \mathbf{x} \mathbf{W}_2 \quad (21)$$

$$\mathbf{z} = f(\mathbf{x}_1) * \mathbf{x}_2 \quad (22)$$

$$\mathbf{y} = \mathbf{z} \mathbf{W}_3, \quad (23)$$

其中， $\mathbf{x}$ 的形状为 (B, D) ， $\mathbf{W}_1$ 和 $\mathbf{W}_2$ 的形状为 (D, D_{FF}) ， $\mathbf{W}_3$ 的形状为 (D_{FF}, D) 。 f 为逐元素激活函数，例如 SiLU， $*$ 表示逐元素乘法。

显式写出反向传播也很有帮助。回忆一下，给定形状为 (B, D) 的 dy ，反向传播为：

$$\text{dz} = \text{dy} \mathbf{W}_3^\top \quad (24)$$

$$\text{dx}_2 = \text{dz} * f(\mathbf{x}_1) \quad (25)$$

$$\text{dx}_1 = \text{dz} * f'(\mathbf{x}_1) * \mathbf{x}_2 \quad (26)$$

$$\text{dx} = \text{dx}_1 \mathbf{W}_1^\top + \text{dx}_2 \mathbf{W}_2^\top \quad (27)$$

$$\text{dW}_3 = \mathbf{z}^\top \text{dy} \quad (28)$$

$$\text{dW}_2 = \mathbf{x}^\top \text{dx}_2 \quad (29)$$

$$\text{dW}_1 = \mathbf{x}^\top \text{dx}_1, \quad (30)$$

其中 $*$ 表示逐元素乘法。

回忆有 N_{DP} 个设备的数据并行：我们将输入 $\mathbf{x}$ 分成大小为 $(B/N_{\text{DP}}, D)$ 的分片 $\mathbf{x}^{(i)}$ 。前向传播照常进行，不需要任何集合通信，产生大小为 $(B/N_{\text{DP}}, D)$ 的激活 $\mathbf{y}^{(i)}$ 。

反向传播同样在按批次分片的激活上照常进行，设备 i 最终得到梯度：

$$d\mathbf{W}_3^{(i)} = (\mathbf{z}^{(i)})^\top d\mathbf{y}^{(i)} \quad (31)$$

$$d\mathbf{W}_2^{(i)} = (\mathbf{x}^{(i)})^\top d\mathbf{x}_2^{(i)} \quad (32)$$

$$d\mathbf{W}_1^{(i)} = (\mathbf{x}^{(i)})^\top d\mathbf{x}_1^{(i)}, \quad (33)$$

这里不是对全部 B 个外积求和，而是只得到当前输入分片对应的 B/N_{DP} 个外积的部分和。因此，还需要跨设备执行 all-reduce，获得完整梯度 $d\mathbf{W}_3, d\mathbf{W}_2, d\mathbf{W}_1$ 。

题目 (data_parallel_calcs)：数据并行计算 (3 分)

现在已经具备计算数据并行何时受到通信瓶颈限制所需的全部条件。令 C （单位 FLOP/s）表示设备的加速器计算速度， W （单位字节/秒）表示每个设备的出口带宽。这样可以计算计算时间和通信时间。由于计算与通信可以重叠，当通信时间大于计算时间时，就会受到通信瓶颈限制。假设所有权重与激活均为 FP16，即每个元素两个字节。

(a) 使用 N_{DP} 路数据并行时，反向传播需要多少 FLOP？可以忽略所有非矩阵乘法操作。回忆矩阵乘法 $(A, B)(B, C) \rightarrow (A, C)$ 需要 $2ABC$ 次浮点运算。

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{DP}}$ 表示答案，并用一句话说明理由。

(b) 使用 N_{DP} 路数据并行时，反向传播需要多少通信时间？

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{DP}}, W$ 中的一部分变量表示答案，并用一句话说明理由。

(c) 固定其他参数， N_{DP} 最多增大到多少，才不会受到通信瓶颈限制？

提交内容：给出一个不等式，一侧为 N_{DP} ，另一侧为由 $B, D, D_{\text{FF}}, C, W$ 中部分变量构成的表达式，并用一句话说明理由。

8.3 全分片数据并行分析

接下来分析 FSDP。回忆一下，与 DP 一样，FSDP 对输入和激活的批维度分片。此外，为了节省显存，还对优化器状态、梯度与权重分片。权重可沿任一维度分片，在设备 i 上形成 $\mathbf{W}_1^{(i)}, \mathbf{W}_2^{(i)}, \mathbf{W}_3^{(i)}$ ，每个分片大小为 $DD_{\text{FF}}/N_{\text{FSDP}}$ 。

前向传播只是对按批次分片的输入执行数据并行前向传播。但在此之前，需要先跨设备 all-gather 权重分片：

$$\mathbf{W}_1 = \text{all-gather}\left(\{\mathbf{W}_1^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (34)$$

$$\mathbf{W}_2 = \text{all-gather}\left(\{\mathbf{W}_2^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (35)$$

$$\mathbf{W}_3 = \text{all-gather}\left(\{\mathbf{W}_3^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (36)$$

$$\text{(执行按批次分片的前向传播)} \quad (37)$$

注意，all-gather 的执行时机有一定自由度：只要在使用权重之前完成 all-gather 即可，使用后应丢弃完整权重以降低显存成本。为简便起见，本节将三次 all-gather 一起列出。

与前向传播一样，反向传播也需要先跨设备 all-gather 权重分片。然后可以进行数据并行反向传播，但不再需要 all-reduce，因为每个设备只需要属于自己分片的梯度。因此改用 reduce-scatter：

$$\mathbf{W}_1 = \text{all-gather}\left(\{\mathbf{W}_1^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (38)$$

$$\mathbf{W}_2 = \text{all-gather}\left(\{\mathbf{W}_2^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (39)$$

$$\mathbf{W}_3 = \text{all-gather}\left(\{\mathbf{W}_3^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (40)$$

$$\text{(执行按批次分片的反向传播)} \quad (41)$$

$$\mathbf{dW}_1^{(i)} = \text{reduce-scatter}\left(\{\mathbf{dW}_1^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (42)$$

$$\mathbf{dW}_2^{(i)} = \text{reduce-scatter}\left(\{\mathbf{dW}_2^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (43)$$

$$\mathbf{dW}_3^{(i)} = \text{reduce-scatter}\left(\{\mathbf{dW}_3^{(i)}\}_{i=0}^{N_{\text{FSDP}}-1}\right) \quad (44)$$

注意，分片记号，即上标 (i) ，在这里有两种含义：reduce-scatter 的输入是完整权重上的部分和，而输出是权重分片上的完整和。

题目 (fsdp_calcs)：全分片数据并行计算 (3 分)

在与数据并行计算题相同的设置下，计算 FSDP 何时受到通信瓶颈限制。

(a) 使用 N_{FSDP} 路 FSDP，反向传播需要多少 FLOP？前向传播呢？

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{FSDP}}$ 表示两个答案，并分别用一句话说明理由。

(b) 使用 N_{FSDP} 路 FSDP，反向传播需要多少通信时间？前向传播呢？

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{FSDP}}, W$ 中的一部分变量表示两个答案，并分别用一句话说明理由。

(c) 固定其他参数， N_{FSDP} 最多增大到多少，反向传播才不会受到通信瓶颈限制？前向传播呢？

提交内容：给出两个不等式，一侧为 N_{FSDP} ，另一侧为由 $B, D, D_{\text{FF}}, C, W$ 中部分变量构成的表达式，并分别用一句话说明理由。

8.4 张量并行分析

实践中，FSDP 经常与称为张量并行（TP）的策略组合使用。在 TP 中，我们跨设备对每个权重矩阵的输入维度或输出维度分片。沿输入维度分片通常称为“行并行”，沿输出维度分片通常称为“列并行”。

具体来说，假设要对矩阵乘法 $\mathbf{x}\mathbf{W}$ 分片，其中 $\mathbf{x}$ 的形状为 (B, D) ， $\mathbf{W}$ 的形状为 (D, D_{FF}) 。在列并行中，分片 $\mathbf{W}^{(i)}$ 的形状为 $(D, D_{\text{FF}}/N_{\text{TP}})$ ，有：

$$\mathbf{x}\mathbf{W} = \text{all-gather}\left(\{\mathbf{x}\mathbf{W}^{(i)}\}_{i=0}^{N_{\text{TP}}-1}\right). \quad (45)$$

另一方面，在行并行中，分片 $\mathbf{W}^{(i)}$ 的形状为 $(D/N_{\text{TP}}, D_{\text{FF}})$ ，而且每个设备在矩阵乘法之前，先将输入 $\mathbf{x}$ 缩窄为形状为 $(B, D/N_{\text{TP}})$ 的切片 $\mathbf{x}^{(i)}$ 。于是：

$$\mathbf{x}\mathbf{W} = \text{all-reduce}\left(\{\mathbf{x}^{(i)}\mathbf{W}^{(i)}\}_{i=0}^{N_{\text{TP}}-1}\right). \quad (46)$$

为了并行化 FFN，我们采用一种特定的张量并行配置： $\mathbf{W}_1$ 和 $\mathbf{W}_2$ 使用列并行，即沿输出维度分片； $\mathbf{W}_3$ 使用行并行，即沿输入维度分片。由于行并行权重只需要输入的一个切片，这种配置可以省去列并行权重计算之后的 all-gather。给定大小为 (B, D) 的输入 $\mathbf{x}$ ，前向传播为：

$$\mathbf{x}_1^{(i)} = \mathbf{x}\mathbf{W}_1^{(i)} \quad (47)$$

$$\mathbf{x}_2^{(i)} = \mathbf{x}\mathbf{W}_2^{(i)} \quad (48)$$

$$\mathbf{z}^{(i)} = f(\mathbf{x}_1^{(i)}) * \mathbf{x}_2^{(i)} \quad (49)$$

$$\mathbf{y}^{(i)} = \mathbf{z}^{(i)}\mathbf{W}_3^{(i)} \quad (50)$$

$$\mathbf{y} = \text{all-reduce}\left(\{\mathbf{y}^{(i)}\}_{i=0}^{N_{\text{TP}}-1}\right), \quad (51)$$

其中 $\mathbf{W}_1^{(i)}$ 与 $\mathbf{W}_2^{(i)}$ 的形状为 $(D, D_{\text{FF}}/N_{\text{TP}})$ ， $\mathbf{W}_3^{(i)}$ 的形状为 $(D_{\text{FF}}/N_{\text{TP}}, D)$ 。

题目 (tp_calcs)：张量并行计算 (4 分)

在与 DP 和 FSDP 计算题相同的设置下，计算 TP 何时受到通信瓶颈限制。

(a) 给定大小为 (B, D) 的输入 $\mathbf{dy}$ ，写出上述张量并行策略的反向传播；其中 $\mathbf{W}_1^{(i)}$ 与 $\mathbf{W}_2^{(i)}$ 的形状为 $(D, D_{\text{FF}}/N_{\text{TP}})$ ， $\mathbf{W}_3^{(i)}$ 的形状为 $(D_{\text{FF}}/N_{\text{TP}}, D)$ 。

提交内容：写出一组描述反向传播的方程，使用 $\mathbf{dy}$ 、分片权重 $(\mathbf{W}_1^{(i)}, \mathbf{W}_2^{(i)}, \mathbf{W}_3^{(i)})$ 、前向传播保存的激活 $(\mathbf{x}, \mathbf{x}_1^{(i)}, \mathbf{x}_2^{(i)}, \mathbf{z}^{(i)}, \mathbf{y}^{(i)})$ 、通信原语，以及你自行定义的中间变量。方程应产生各设备的梯度 $d\mathbf{W}_1^{(i)}, d\mathbf{W}_2^{(i)}, d\mathbf{W}_3^{(i)}$ ，以及反向传播输出 $d\mathbf{x}$ 。可以参考并修改第 8.2 节的未分片反向传播。

(b) 使用 N_{TP} 路 TP 时，前向传播需要多少 FLOP？反向传播呢？

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{TP}}$ 表示两个答案，并分别用一句话说明理由。

(c) 使用 N_{TP} 路 TP 时，前向传播需要多少通信时间？反向传播呢？

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{TP}}, W$ 中的一部分变量表示两个答案，并分别用一句话说明理由。

(d) 固定其他参数， N_{TP} 最多增大到多少，反向传播才不会受到通信瓶颈限制？前向传播呢？

提交内容：给出两个不等式，一侧为 N_{TP} ，另一侧为由 $B, D, D_{\text{FF}}, C, W$ 中部分变量构成的表达式，并分别用一句话说明理由。

8.5 二维并行 (FSDP + TP)

终于可以组合并行策略了！本节将研究如何最优地组合 FSDP 与 TP。提示：在前几节中，你应该已经发现，批大小和模型规模参数会限制能够扩展到的设备数量；把这些规模都增大，就可以继续扩展设备而不受到通信瓶颈限制。不幸的是，批大小超过某个点后，会开始降低性能，因为梯度噪声显著减小，丢失了 SGD 的隐式正则化性质；这个点通常称为“临界批大小”。而缩放定律往往告诉我们模型应该多大，这将是你在下次作业中的任务！

本节考虑一个简化情境：有人给出全部问题参数，包括批大小、模型规模、带宽和加速器计算速度。你的任务是选择 FSDP 与 TP 的配置，在保持计算受限而非通信受限的条件下，扩展到尽可能多的设备。

先梳理 FSDP 与 TP 组合的机制。每个设备都有一个 TP rank $i = 0, \dots, N_{\text{TP}} - 1$ ，和一个 FSDP rank $j = 0, \dots, N_{\text{FSDP}} - 1$ ，组成一个共包含 $N = N_{\text{TP}} N_{\text{FSDP}}$ 个设备的二维网格。按照 TP，先沿输出维度对 $\mathbf{W}_1, \mathbf{W}_2$ 分片，并沿输入维度对 $\mathbf{W}_3$ 分片。因此，需要在激活上插入 TP 风格的 all-reduce。接着应用 FSDP：对输入的批维度分片，并沿每个权重矩阵尚未被 TP 分片的维度，进一步对该矩阵分片。随后，在 TP 风格的前向/反向传播之前，插入 FSDP 风格的权重 all-gather；在 TP 风格的反向传播之后，对权重梯度执行 reduce-scatter。

因此，每个设备 (i, j) 持有形状为 $(D/N_{\text{FSDP}}, D_{\text{FF}}/N_{\text{TP}})$ 的权重分片 $\mathbf{W}_1^{(i,j)}, \mathbf{W}_2^{(i,j)}$ ，以及形状为 $(D_{\text{FF}}/N_{\text{TP}}, D/N_{\text{FSDP}})$ 的权重分片 $\mathbf{W}_3^{(i,j)}$ 。给定大小为 $(B/N_{\text{FSDP}}, D)$ 、按批次分片的输入 $\mathbf{x}^{(j)}$ ，前向传播可写为：

$$\mathbf{W}_1^{(i)} = \text{all-gather}\left(\{\mathbf{W}_1^{(i,j)}\}_{j=0}^{N_{\text{FSDP}}-1}\right) \quad (52)$$

$$\mathbf{W}_2^{(i)} = \text{all-gather}\left(\{\mathbf{W}_2^{(i,j)}\}_{j=0}^{N_{\text{FSDP}}-1}\right) \quad (53)$$

$$\mathbf{W}_3^{(i)} = \text{all-gather}\left(\{\mathbf{W}_3^{(i,j)}\}_{j=0}^{N_{\text{FSDP}}-1}\right) \quad (54)$$

$$\mathbf{x}_1^{(i,j)} = \mathbf{x}^{(j)} \mathbf{W}_1^{(i)} \quad (55)$$

$$\mathbf{x}_2^{(i,j)} = \mathbf{x}^{(j)} \mathbf{W}_2^{(i)} \quad (56)$$

$$\mathbf{z}^{(i,j)} = f(\mathbf{x}_1^{(i,j)}) * \mathbf{x}_2^{(i,j)} \quad (57)$$

$$\mathbf{y}^{(i,j)} = \mathbf{z}^{(i,j)} \mathbf{W}_3^{(i)} \quad (58)$$

$$\mathbf{y}^{(j)} = \text{all-reduce}\left(\{\mathbf{y}^{(i,j)}\}_{i=0}^{N_{\text{TP}}-1}\right), \quad (59)$$

最终得到大小为 $(B/N_{\text{FSDP}}, D)$ 的按批次分片输出 $\mathbf{y}^{(j)}$ 。为简洁起见，本节省略反向传播，仅关注前向传播。但到这里，你应该已经具备自行写出反向传播所需的全部信息。

题目 (fsdp_tp_calcs): 二维并行计算 (6 分)

在与此前计算相同的设置下，计算二维并行何时遇到瓶颈。

(a) 使用 N_{FSDP} 路 FSDP 加 N_{TP} 路 TP 时，前向传播需要多少 FLOP?

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{FSDP}}, N_{\text{TP}}$ 表示答案，并用一句话说明理由。

(b) 使用 N_{FSDP} 路 FSDP 加 N_{TP} 路 TP 时，前向传播需要多少通信时间？假设各轴上的通信可以重叠，也就是说 FSDP 轴上的集合通信能与 TP 轴上的集合通信重叠。

提交内容：用 $B, D, D_{\text{FF}}, N_{\text{FSDP}}, N_{\text{TP}}, W$ 中的一部分变量表示答案，并用一句话说明理由。提示：由于两者可以重叠，答案应表示为两个量的最大值，即 FSDP 与 TP 的集合通信成本的 max。

(c) 在 N_{TP} 与 N_{FSDP} 取最优配置时， $N = N_{\text{TP}}N_{\text{FSDP}}$ 最多能增大到多少，前向传播才不会受到通信瓶颈限制？

提交内容：给出一个不等式，一侧为 N ，另一侧为由 $B, D, D_{\text{FF}}, C, W$ 中部分变量构成的表达式，并用几句话与若干方程说明理由。

(d) 现在假设 FSDP 轴与 TP 轴的集合通信共享同一网络资源，因此不能重叠。在 N_{TP} 与 N_{FSDP} 的最优配置下， $N = N_{\text{TP}}N_{\text{FSDP}}$ 最多能增大到多少，前向传播才不会受到通信瓶颈限制？不必考虑把 N_{TP} 与 N_{FSDP} 截断为整数。

提交内容：给出一个不等式，一侧为 N ，另一侧为由 $B, D, D_{\text{FF}}, C, W$ 中部分变量构成的表达式，并用几句话与若干方程说明理由。

9 排行榜

作业 2 的排行榜将测试一个 8B 模型完整训练步骤的速度。我们鼓励你对代码进行基准测试，并运用能想到的技巧来优化显存和运行时间。主要限制是：不能改变模型的输入／输出行为。你的实现会与 cs336_basics 目录中的模型进行对照测试。输入以 BF16 精度并启用因果掩码进行测试，必须通过你的常规实现相同的测试。此外，实现必须由你自行完成，不能使用或复制已有实现。应在两个 B200 GPU 上，使用批大小为 2、序列长度为 32,768 的样本测量耗时。我们有意让模型难以装入显存。完整配置见下方代码。

我们将验证排名前 5–10 的提交的正确性与性能。用于测量你的实现耗时的测试如下：

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2
cfg = Config()
def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size,
cfg.ctx_len))
    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())
    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()
    timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
    print(timing_results)
```

测试时，可以将重复测量和预热时间（单位为毫秒）缩短。

以下是一些改进性能并确保模型能够装入显存的思路：

- 调优内核分块大小；可以使用 Triton autotune!
- 调整更多 Triton / torch.compile 配置参数。
- 实现融合版 AdamW。
- 基础实现会生成完整 logits，形状为 [batch, seq_len, vocab_size]。编写一个融合 LM head 与交叉熵损失的内核。也可以让它立即以融合方式计算反向传播。
- 改进 FlashAttention：
 - 使用 Triton 实现反向传播，而不只是 torch.compile；参见第 4.2.3 节。
 - 在反向传播中对输入处理两遍，一遍用于 dQ，另一遍用于 dK 和 dV，避免原子操作或块间同步。
 - 使用因果掩码时，提前结束程序实例，跳过确定全部为零的分块。
 - 将无需掩码的分块与位于分块对角线的分块分开：前者完全不比较索引，后者只做一次比较。

▷ 在 Hopper 之后的架构上使用 TMA (Tensor Memory Accelerator, 张量内存加速器) 功能, 采用与我们的教程类似的模式。

- 只有确有需要时, 才使用激活检查点, 以运行速度换取显存节约。

题目 (leaderboard): 排行榜: 最快训练步骤 (10 分)

基准测试将在两个 B200 GPU 上、以批大小 2 运行。提交将按完整训练步骤的实际耗时进行评估, 包括前向传播、损失计算、反向传播和 AdamW 更新。

从空的 PyTorch / Triton 缓存开始, 基准测试必须在 10 分钟内完成, 因此要谨慎使用过于激进的 `torch.compile` 与 Triton 自动调优。

提交内容: 包含 AdamW 的完整前向加反向训练步骤所达到的最佳实际耗时。

我们期望排行榜提交能够超过 10 秒的朴素基线。

请在这里向排行榜提交结果:

<https://github.com/stanford-cs336/assignment2-systems-leaderboard>

参考文献

- [1] T. Dao, 《FlashAttention-2: 通过更好的并行性与工作划分加速注意力》(FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning).[在线] 可获取: <https://arxiv.org/abs/2307.08691>。
- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Re, 《FlashAttention: 具有 IO 感知能力的快速、显存高效的精确注意力》(FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness), 载于《神经信息处理系统进展》(Advances in Neural Information Processing Systems), A. H. Oh, A. Agarwal, D. Belgrave 和 K. Cho 编, 2022。[在线] 可获取: <https://openreview.net/forum?id=H4DqfPSibmx>。
- [3] M. Milakov, N. Gimershein, 《Softmax 的在线归一化因子计算》(Online normalizer calculation for softmax)。<[在线] 可获取: <https://arxiv.org/abs/1805.02867>。
- [4] H. He, 《从第一性原理出发, 让深度学习飞速运行》(Making Deep Learning Go Brrrr From First Principles), 2022。[在线] 可获取: https://horace.io/brrr_intro.html。
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, 《ZeRO: 面向万亿参数模型训练的显存优化》(ZeRO: Memory Optimizations Toward Training Trillion Parameter Models), 2020。
- [6] J. Austin 等, 《如何扩展你的模型》(How to Scale Your Model), 2025。
- [7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, 《超大规模训练手册: 在 GPU 集群上训练大语言模型》(The Ultra-Scale Playbook: Training LLMs on GPU Clusters), 2025。